COURS PAR CORRESPONDANCE

ANALYSTE EN INFORMATIQUE

Initiation au système d'exploitation UNIX

Module

Systèmes, réseaux & SSI

Initiation au système d'exploitation UNIX

(Lecture obligatoire)

Table des matières

1 Introduction **3**
- 1.1 Objectifs de ce document . . . 3
- 1.2 Quelques mots sur les systèmes d'exploitation . . . 3
- 1.3 Environnement de travail . . . 4
- 1.4 Contenu de ce document . . . 4

2 Prise en main **5**
- 2.1 Ouverture de session . . . 5
- 2.2 Utilisation du Terminal . . . 5
- 2.3 Commandes élémentaires . . . 6
  - 2.3.1 Man est ton ami . . . 6
  - 2.3.2 La commande echo . . . 6
  - 2.3.3 La commande who . . . 6
  - 2.3.4 La commande expr . . . 6

3 Les fichiers **8**
- 3.1 Fichiers et système de fichiers . . . 8
- 3.2 Comparaison entre fichiers et mémoire RAM . . . 8
- 3.3 Les fichiers sous UNIX . . . 9
  - 3.3.1 L'arborescence de fichiers sous UNIX . . . 9
  - 3.3.2 Liens et fichiers . . . 10
  - 3.3.3 Les méta-données sur les fichiers UNIX . . . 10
  - 3.3.4 Les répertoires principaux dans les systèmes UNIX . . . 10
  - 3.3.5 Connaître le contenu d'un répertoire . . . 11
  - 3.3.6 Parcourir le système de fichier . . . 11
  - 3.3.7 Création d'une arborescence de fichiers . . . 11
  - 3.3.8 Modification d'une arborescence de fichiers . . . 12
  - 3.3.9 Les droits d'accès des fichiers UNIX . . . 13

4 Les processus **15**
- 4.1 Du programme vers le processus . . . 15
- 4.2 Partage des ressources . . . 16
  - 4.2.1 Partage du processeur . . . 16
  - 4.2.2 Partage de la mémoire . . . 16
- 4.3 Organisation des processus . . . 17
- 4.4 États des processus . . . 17
- 4.5 Interruption, mode utilisateur et mode système . . . 18
- 4.6 Les entrées/sorties . . . 19
- 4.7 Création de processus . . . 20

4.8 Terminaison d'un processus . . . 21
4.9 Communication inter-processus . . . 21
4.9.1 Les signaux . . . 21
4.9.2 Les fichiers . . . 23
4.9.3 Les tubes . . . 23
4.9.4 Autres solutions existantes . . . 24

5 Shell-script **25**
5.1 Le shebang . . . 25
5.2 Commentaire . . . 25
5.3 Les variables . . . 25
5.3.1 Lecture et écriture . . . 25
5.3.2 Les variables spéciales . . . 26
5.3.3 Les variables d'environnement . . . 26
5.4 Les tests . . . 26
5.5 Les quotes . . . 27
5.6 Expression numérique . . . 27
5.7 Les structures de contrôle . . . 28
5.7.1 Structure conditionnel . . . 28
5.7.2 Les boucles . . . 28

6 Exécution de commande **30**
6.1 Fichier de la commande . . . 30
6.2 Processus et commande . . . 31

7 Synchronisation de processus **33**
7.1 Ressources partagées et concurrence . . . 33
7.2 Section critique et verrous . . . 35
7.3 Synchronisation en shell . . . 36
7.4 Synchronisation à base de sémaphores . . . 38

A Aide mémoire shell **41**

B Guide d'installation de Linux Ubuntu sur un ordinateur personnel **44**
B.1 Téléchargement de l'image Iso . . . 44
B.2 Installation native de Ubuntu en double boot . . . 44
B.3 Installation de Ubuntu dans une machine virtuelle . . . 45
B.4 Installation des logiciels de base sur Ubuntu . . . 46

Chapitre 1

Introduction

1.1 Objectifs de ce document

Ce document a pour objectif d'initier ou de réviser l'utilisation des systèmes UNIX au préalable des séances présentielles. Il présente d'un point de vue macroscopique les différents concepts des systèmes d'exploitation et ceux propres à UNIX à travers un langage de commande : le shell.

N.B. : Ce document se base sur le livre *UNIX : programmation et communication de J-M Rifflet et de J-B Yunès, Dunod 2003* et notamment les chapitres 1, 2, 3 et 4.

1.2 Quelques mots sur les systèmes d'exploitation

Le système d'exploitation est une couche logicielle permettant de faire l'interface entre le matériel et les applications des utilisateurs de la machine. Il est notamment responsable de la gestion des ressources de la machine comme le processeur, la mémoire et les périphériques comme les disques dur, les cartes réseaux (Wifi, Ethernet), la carte graphique, la carte son, etc.

UNIX est un des premiers systèmes d'exploitation (OS) apparu dans les années 70 et fut le système d'exploitation le plus utilisé jusqu'à la fin des années 80. Le développement d'une multitude de versions propriétaires pour différentes architectures et l'émergence au début des années 90 de Windows, plutôt adapté aux utilisateurs non-informaticiens, ont causé une énorme perte de popularité de UNIX. De nos jours UNIX désigne surtout une famille de systèmes d'exploitation plutôt qu'un OS à part entière. On note donc deux familles de systèmes d'exploitation : Windows et UNIX. Parmi les OS UNIX qui ont résisté à la popularité de Windows, on peut citer BSD, Linux ou encore MacOS X qui reprennent du plus en plus d'ampleur dans le grand public.

Une distribution Linux est une collection de logiciels pour la plupart open-source composée du système d'exploitation Linux (on parle aussi de noyau Linux) et d'une suite de librairies, d'utilitaires ou applications. Chaque distribution est maintenue par une communauté de développeurs permettant ainsi de produire des mises à jour régulières des différents logiciels associés à la distribution. En revanche, le noyau Linux est développé indépendamment de toute distribution. Il constitue donc un point commun entre elles.

Parmi les distributions existantes, on note des distributions mères (ArchLinux, Debian, Gentoo, RedHat, Slackware, Android) qui se différencient par exemple par les outils utilitaires de gestion de paquets (aptitude pour Debian, Rpm pour RedHat, ...). De ces distributions mères ont été développées des distributions filles qui fournissent les logiciels de leur distribution mère associée et d'autres utilitaires ou applications qui leur sont propres.

1.3 Environnement de travail

Des postes de travail sous Debian seront mis à disposition lors des séances présentielles . Un guide d'installation de la distribution Ubuntu est cependant fourni en annexe B de ce document si le lecteur souhaite utiliser Linux sur son ordinateur personnel.

1.4 Contenu de ce document

Ce document abordera dans un premier temps la manière de prendre en main un ordinateur en ligne de commande, puis décrira les notions de fichiers et de système de fichiers ainsi que les commandes de manipulation associées. On abordera ensuite la notion fondamentale de processus et enfin le scripts en shell qui permettront de réaliser les différents travaux pratiques.

Chapitre 2

Prise en main

2.1 Ouverture de session

Comme la plupart des systèmes d'exploitation, Linux est un système multi-utilisateurs c'est à dire qu'il permet à plusieurs utilisateurs d'utiliser l'ordinateur simultanément tout en limitant les droits d'accès de chacun afin de garantir l'intégrité de leurs données. Pour pouvoir utiliser l'ordinateur, un utilisateur doit se connecter à la machine en ouvrant une session d'utilisation. Lors de la connexion, l'utilisateur doit renseigner :

- **un login** permettant de s'identifier de manière unique
- **un moyen d'authentification** permettant d'authentifier la connexion, c'est à dire de prouver que l'utilisateur est bien qui il prétend être. Il est possible de s'authentifier par **mot de passe** ou par **clé**.

L'ouverture d'une session peut se faire de manière :

- **locale** en se connectant physiquement sur la machine
- **distante** depuis une autre machine grâce à un réseau. Ceci permet par exemple d'administrer ou d'utiliser un ensemble de machines depuis un unique poste de travail.

Une fois connecté(e), votre bureau de session d'affiche ainsi qu'un dock sur le coté gauche de l'écran avec des liens vers différentes applications installées sur la machine. Pour utiliser un terminal dans la session graphique, il faut utiliser l'application "Terminal".

2.2 Utilisation du Terminal

L'utilisation et l'administration d'un ordinateur peuvent se faire de manière graphique grâce à des fenêtres et à l'utilisation d'une souris (comme le font la plupart des utilisateurs) ou bien en mode texte à travers une console ou un terminal où les commandes sont directement données au clavier. Lorsqu'on lance l'application "Terminal" , le système ouvre une fenêtre puis exécute un programme (ou processus) particulier appelé **interpréteur de commande** ou **shell** reconnaissable par la présence d'une chaîne de caractères appelée **invite** ou **prompt**. Ceci invite l'utilisateur à entrer des commandes de son choix. Le prompt par défaut se termine par un caractère comme $, `%` ou `#` précédé par une chaîne de caractères tel que le login de l'utilisateur et le nom de la machine séparés par un `@`. L'invocation d'une commande consiste à écrire le **nom** de cette commande, suivi d'un ou plusieurs autres mots appelés **arguments** ou **paramètres** et de taper ensuite sur la touche *Entrée* :

$$nom_commande\ [argument_1\ ...\ argument_n]$$

Les différentes chaînes de caractères constituant le nom et les arguments d'une commande sont séparées par au moins un caractère d'espacement. Les arguments d'une commande peuvent être :

- des **paramètres optionnels** ou **options** permettant d'obtenir des variantes de comportement de la commande : ils sont, le plus souvent, précédés du caractère − ou de la chaîne −−.
- des **arguments ordinaires** sur lesquels la commande doit être appliquée (par exemple des noms de fichiers, d'utilisateurs, un nombre, etc.)

Après lecture d'une ligne, l'interpréteur shell l'analyse, l'exécute et ré-affiche le prompt une fois que la commande se termine pour permettre à l'utilisateur de saisir la commande suivante. Lorsque le shell détecte une erreur dans la commande (par exemple : commande introuvable ou erreur de paramètre) il affiche un message d'erreur.

2.3 Commandes élémentaires

2.3.1 Man est ton ami

Sous Unix il existe la commande d'aide ***man*** qui permet d'afficher sur le terminal la notice d'utilisation d'une commande. Pour plus d'information sur cette commande, tapez :

```
man man
```

Pour sortir d'une page d'un manuel, tapez la lettre `q`.

2.3.2 La commande echo

La commande ***echo*** permet d'afficher sur la sortie standard chacun de ses arguments. Par exemple, pour afficher *Bonjour tout le monde* sur le terminal, il faut taper la commande suivante :

```
echo "Bonjour tout le monde"
```

ou alors

```
echo Bonjour tout le monde
```

On pourra remarquer que la commande ***echo*** ajoute implicitement un caractère de retour chariot à la fin (ce qui fait que le prompt de retour s'affiche toujours à la ligne suivante). Pour éviter de passer à la ligne il faut renseigner l'option `-n` :

```
echo -n "Bonjour tout le monde"
```

2.3.3 La commande who

La commande ***who*** permet principalement de lister les utilisateurs qui sont actuellement connectés à la machine. Il est également possible d'afficher des informations complémentaires sur le système avec l'option `-a`

2.3.4 La commande expr

La commande ***expr*** permet d'évaluer des expressions sur des entiers. Il est également possible de faire des opérations sur les chaînes de caractères. En ce qui concerne les opérations entières, 3 arguments sont attendus :

- le premier argument correspond à la première opérande

- le deuxième argument correspond à l'opérateur
- le troisième argument correspond à la deuxième opérande

L'opérateur peut être

- `+` pour l'addition
- `-` pour la soustraction
- `*` pour la multiplication. Attention le caractère `*` étant un méta-caractère du shell il est nécessaire de protéger ce caractère avec des simples quotes (`'`) ou bien en le précédent d'un \ si on souhaite que le shell l'interprète comme un caractère classique.
- `/` pour la division entière
- `%` pour le reste de la division entière
- `<` ou `<=` pour tester si la valeur de opérande 1 est inférieure ou inférieure ou égale à celle de l'opérande 2
- `>` ou `>=` pour tester si la valeur de l'opérande 1 est supérieure ou supérieure ou égale à la valeur de l'opérande 2
- `=` pour tester si valeur de l'opérande 1 est égale à celle de l'opérande 2
- `!=` pour tester si valeur de l'opérande 1 est différente de celle de l'opérande 2

On trouvera un complément à ce sujet dans la section 5.6

Chapitre 3

Les fichiers

3.1 Fichiers et système de fichiers

Classiquement, un fichier représente un ensemble de données que l'on stocke de manière persistante sur un volume stable (disque dur, clé USB, ...). Ceci permet aux utilisateurs de retrouver leurs données même après extinction de l'ordinateur. Un fichier s'inscrit dans un système de fichiers. Le système de fichier définit :

- d'un point de vue logique une organisation hiérarchique des fichiers formant ainsi une arborescence. Cette organisation logique sera propre au système d'exploitation : par exemple sous UNIX il n'existe qu'une seule racine impliquant une unique arborescence pour tous les volumes alors que sous Windows, ou pourra remarquer que chaque volume est représenté par une lettre (`C:`, `D:` ...) impliquant ainsi une arborescence par volume.
- d'un point de vue physique la façon dont les données sont organisées physiquement sur le volume. Cette organisation physique peut inclure des mécanismes de journalisation (pour gérer les erreurs par exemple) et diffère selon le système de fichiers choisi lors du formatage du volume. Par exemple le système de fichiers le plus répondu est NTFS pour Windows et Ext4 pour Linux.

3.2 Comparaison entre fichiers et mémoire RAM

Contrairement aux données stockées dans un fichier, les données stockées dans la mémoire RAM sont (à l'heure actuelle) non persistantes, c'est à dire qu'elles sont perdues dès l'extinction de la machine qu'elle soit voulue ou non. En revanche le temps d'accès d'une donnée sur la RAM est considérablement plus petit qu'un accès disque (de l'ordre d'un facteur 1000). De plus la granularité d'accès est beaucoup plus fine sur la mémoire RAM que sur un disque. En effet l'accès aux données sur un disque se fait par bloc de données contigu physiquement (dont la taille est définie lors du formatage) alors que la RAM permet d'accéder directement à un octet donné (un octet est identifié par une adresse). Par conséquent, vouloir accéder à une donnée particulière sur un fichier nécessite de charger dans un premier temps le bloc contenant cette donnée en RAM et dans un second temps à la donnée voulue. Lire ou écrire dans un fichier nécessitent donc des mécanismes beaucoup plus complexes que les accès RAM. Il faut également noter qu'il est impossible d'allouer partiellement un bloc disque, ainsi la taille réelle d'un fichier (c'est à dire le nombre de données en octets que le fichier contient) est différente de sa taille physique qui est forcément un multiple de la taille d'un bloc.

3.3 Les fichiers sous UNIX

La notion de fichier sous UNIX ne se résout pas uniquement au fichier disque classique. Il permet également de désigner une ressource physique ou logique de l'ordinateur (appelé **device**), comme par exemple un terminal, une imprimante, un disque physique ou logique (=**partition** du disque physique), la mémoire vive du système, un bus, une carte réseau, etc. Un fichier est donc caractérisé par son **type** qui définit l'ensemble des opérations que l'on peut lui appliquer. On peut classer les types de fichiers en deux grandes catégories :

- **fichiers disques** comme les fichiers réguliers, les répertoires, les liens symboliques ou les tubes nommés.
- **fichiers spéciaux** qui désignent une ressource du système

3.3.1 L'arborescence de fichiers sous UNIX

Les fichiers réguliers permettent de stoker des codes de programmes (binaires ou sources) ou bien des données classiques. Les répertoires sont des fichiers disques particuliers car ils permettent d'une part d'organiser le système de fichiers en arborescence et d'autre part de permettre de désigner de manière unique un lien vers un fichier du système. Un lien est un nom de fichier. Sous UNIX, on retrouve bien souvent l'arborescence de la figure 3.1.

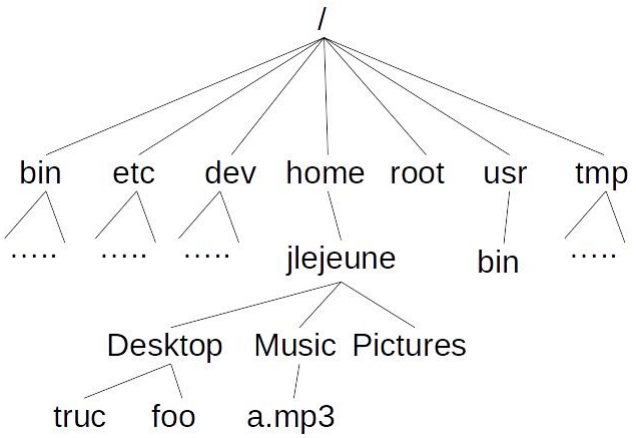


FIGURE 3.1 – Arborescence d'un système de fichiers UNIX

Le dossier le plus haut dans l'arborescence est le dossier racine et se désigne par /. Un nom de fichier (ou `lien`) se désigne sans ambiguïté grâce au chemin qu'il faut parcourir depuis la racine du système pour l'atteindre, c'est le **chemin absolu**. Par exemple le lien foo se désigne par le chemin absolu suivant :

```
/home/jlejeune/Desktop/foo
```

Il est remarquable qu'il existe deux liens qui se nomment *bin*. Cependant, ce sont deux liens différents, l'un se désigne par */bin* et l'autre par */usr/bin*.

Dans un terminal, le shell est associé à un répertoire courant. Par défaut le répertoire courant du shell est la racine de votre répertoire personnel qui se désigne par $\sim$. La commande ***pwd*** affiche le chemin absolu du répertoire courant.

La notation de chemin absolu peut devenir très lourde notamment lorsque le lien à atteindre est très profond dans l'arborescence. C'est pourquoi il est possible de désigner des liens de manière relative (**chemin relatif**) au répertoire courant du shell. Par exemple, si le répertoire courant est */home/jlejeune* alors il est possible de désigner le lien foo par le chemin relatif *Desktop/foo*.

3.3.2 Liens et fichiers

Lien physique

Dans le système d'exploitation, un fichier est identifié sans ambiguïté par deux informations :
- un entier appelé i-nœud
- l'identifiant de volume sur lequel le fichier est stocké

D'un point de vue humain, il est plus facile et plus lisible d'utiliser des noms pour les désigner (tout volume et tout type confondu). Le nom d'un fichier est appelé **lien physique**. Comme nous l'avons vu précédemment, les liens sont organisés de manière hiérarchique et forme une arborescence. Un lien pouvant avoir des liens fils désigne donc un i-nœud de type répertoire. Cependant le système permet qu'un fichier (= un i-nœud) puisse avoir plusieurs liens physiques. Il est donc possible d'avoir deux liens dans l'arborescence qui désignent le même fichier physique. Ceci peut avoir plusieurs effets de bord. Supposons qu'un fichier possède les liens physiques x et y :
- toute modification sur le fichier à partir du lien x sera répercutée sur le lien y et réciproquement
- le fichier existera tant qu'il existe au moins un lien dans l'arborescence : effacer un lien n'implique donc pas forcément que le fichier sous-jacent soit réellement effacé.

Par abus de langage, on considère la plupart du temps qu'un lien donné est équivalent à un fichier donné car le nombre de liens physiques pour un fichier est dans 99% des cas égal à 1. Pour créer un deuxième lien physique sur un lien existant il faut utiliser la commande ***ln***.

Lien symbolique

Pour assurer d'avoir un seul lien physique par fichier tout en ayant la possibilité de le désigner à plusieurs endroits dans l'arborescence, il est possible d'utiliser des liens symboliques. Un **lien symbolique** est un type de fichier qui permet de référencer un lien cible dans l'arborescence. Contrairement au lien physique, un lien symbolique est donc un fichier à part entière (donc un i-nœud à part entière) qui n'est pas lié physiquement au fichier que l'on veut désigner. Ceci peut être vu comme un raccourci. Par conséquent, si le lien cible est supprimé, le lien symbolique sera toujours présent mais il sera possible de détecter qu'il référence quelque chose qui n'existe plus. On supprime ainsi les effets de bords indésirables des multiples liens physiques. Pour créer un lien symbolique sur un lien existant il faut utiliser la commande ***ln*** avec l'option `-s`.

3.3.3 Les méta-données sur les fichiers UNIX

En tapant la commande ***stat*** avec un nom de fichier comme argument, il est possible d'avoir des informations sur ce fichier maintenues par le système notamment :
- les deux informations identifiant un fichier : l'i-nœud et le numéro de volume.
- sa taille réelle en octet
- le nombre de liens physiques
- le type du fichier (fichier régulier, répertoire, lien symbolique, fichier spécial, etc.)
- les dates d'accès, de modification et de création.

3.3.4 Les répertoires principaux dans les systèmes UNIX

À titre informatif, voici les principaux répertoires que l'on retrouve dans les systèmes UNIX :
- **/bin** et **/usr/bin** : contient les codes des commandes de bases du système.
- **/etc** : contient une sous-arborescence des fichiers de configuration système
- **/dev** : contient la sous-arborescence des fichiers spéciaux (les ressources)

- /**home** : contient les différentes arborescences des répertoires contenant les données des utilisateurs
- /**root** : répertoire du super-utilisateur dont le login est *root*. C'est l'utilisateur qui dispose de tous les droits pour les activités d'administration du système.
- /**usr** : contient une sous-arborescence où sont installés les programmes utilisateurs (plus ou moins l'équivalent du répertoire C : $\backslash programmes$ sous Windows)
- /**tmp** : répertoire contenant des fichiers temporaires utilisés/créés par les applications. Le contenu de ce répertoire est bien souvent effacé au moment du démarrage du système.

3.3.5 Connaître le contenu d'un répertoire

Pour lister les fichiers d'un répertoire il faut utiliser la commande ***ls***. Ainsi :
- `ls` affiche les fichiers du répertoire courant
- `ls /` affiche les fichiers de la racine de l'arborescence
- `ls ~` affiche les fichiers du dossier personnel de l'utilisateur courant

On peut noter les options suivantes :
- `-a` pour afficher tous les fichiers y compris les fichiers cachés (le nom d'un fichier caché commence par un point).
- `-l` pour afficher davantage d'informations sur les fichiers.

3.3.6 Parcourir le système de fichier

On peut remarquer lorsque l'on tape la commande `ls -a rep` que deux fichiers aux noms particuliers se sont affichés : . et ... Il correspondent respectivement au répertoire `rep` et au répertoire parent de `rep`. Ainsi il est toujours possible de désigner le répertoire courant par . et le répertoire parent du répertoire courant par ..

Il est possible de changer de répertoire courant grâce à la commande ***cd***. En tapant :
- `cd ..` le nouveau répertoire courant deviendra le parent du répertoire courant actuel
- `cd rep` le nouveau répertoire courant sera `rep`
- `cd` ou `cd ~` permet de prendre comme répertoire courant le dossier personnel de l'utilisateur

3.3.7 Création d'une arborescence de fichiers

Pour créer une arborescence, nous avons besoins de deux commandes :
- la commande ***mkdir*** qui permet de créer un répertoire vide.
- la commande ***touch*** qui permet de changer la date de modification d'un fichier à la date courante ou créer un fichier régulier si celui-ci n'existe pas.

Pour créer dans le répertoire personnel l'arborescence de la figure 3.2 (les fichiers réguliers sont en gras et entourés). Il faut donc taper les commandes suivantes :

```
mkdir -p essai/B/foo/bar
mkdir -p essai/B/bar
touch essai/A
touch essai/bar
touch essai/B/foo/bar/a.txt
touch essai/B/foo/bar/foo
```

Pour information l'option `-p` de la commande `mkdir` permet de créer les répertoires intermédiaires (à savoir `B` et `foo`) si ces derniers n'existent pas.

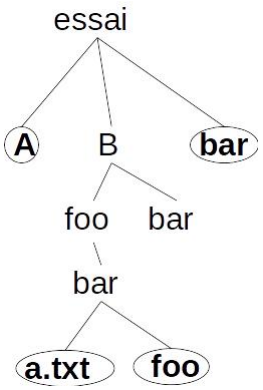


FIGURE 3.2 – Arborescence à créer

Il vous est possible de contrôler que l'arborescence correspond bien à celle attendu en tapant la commande `ls -R` qui permet d'afficher récursivement le contenu de tous les sous-répertoires. Vous avez la possibilité d'utiliser la commande `tree` si cette dernière est installée.

3.3.8 Modification d'une arborescence de fichiers

Il est possible de modifier une arborescence :

- soit en créant un nouveau fichier à partir d'une copie d'un fichier déjà existant avec la commande ***cp***.
- soit en déplaçant un fichier existant vers une autre branche de l'arborescence avec la commande ***mv***.
- soit en supprimant un lien avec la commande ***rm*** ou bien ***rmdir*** pour les dossiers vides.

On notera que déplacer un fichier est fondamentalement différent que de copier le fichier et de supprimer l'orignal ensuite. En effet, la copie implique la création d'un nouveau fichier (donc d'un nouveau i-nœud), de copier octet par octet les données du fichier source vers le fichier cible et ensuite de supprimer le lien du fichier original. Le déplacement est beaucoup plus simple (et donc moins coûteux en nombre d'opérations) car il nécessite juste de changer le nom du lien pour un même i-nœud à condition bien entendu que l'arborescence cible se trouve sur le même volume physique que l'emplacement original. La commande `mv` peut donc être vu comme une commande de renommage de fichier.

Dans notre exemple on pourra :

- copier le fichier $a.txt$ dans le répertoire $essai/B/bar$ à l'aide de la commande
  `cp essai/B/foo/bar/a.txt essai/B/bar`
- déplacer dans le répertoire $essai/B/foo$ la copie du fichier $a.txt$ que vous venez de mettre dans le répertoire $essai/B/bar$ à l'aide de la commande
  `mv essai/B/bar/a.txt essai/B/foo/`
- effacer le lien $essai/B/foo/a.txt$ à l'aide de la commande
  `rm essai/B/foo/a.txt`

- copier le dossier $B/foo/bar$ dans le répertoire B/bar à l'aide de la commande
  `cp -r essai/B/foo/bar essai/B/bar/`
  On notera que l'option `-r` est obligatoire pour pouvoir copier récursivement la sous-arborescence d'un répertoire.
- effacer la copie du dossier que vous venez de faire l'aide de la commande

`rm -r essai/B/bar/bar`

Sur le même principe que `cp`, l'option `-r` est obligatoire pour pouvoir supprimer récursivement la sous-arborescence d'un répertoire.

Attention : On notera que la commande `rm` supprime définitivement un fichier (sauf si ce dernier possède un autre lien physique dans l'arborescence du système). En mode terminal il n'y a donc aucun répertoire intermédiaire type "corbeille" permettant de récupérer une éventuelle suppression par erreur. La commande `rm` est donc à manipuler avec prudence.

3.3.9 Les droits d'accès des fichiers UNIX

Les différents fichiers (tout type confondu) sont associés à un utilisateur propriétaire. Afin d'éviter qu'un utilisateur n'accède à des fichiers dont il n'est pas propriétaire et sans autorisation, le système permet de définir des droits d'accès pour chaque fichier.

Du point de vue d'un fichier donné, un utilisateur quelconque appartient à l'une des trois catégories suivantes :

- le **propriétaire**
- le **groupe d'utilisateurs auquel le propriétaire appartient** (le propriétaire exclu)
- les **autres utilisateurs** (le propriétaire et le groupe du propriétaire exclus)

On peut effectuer trois types d'opération sur un fichier :

- la **lecture** du contenu du fichier (identifié par le caractère `r`)
- l'**écriture** dans le fichier (identifié par le caractère `w`)
- l'**exécution**, c'est à dire pouvoir interpréter le fichier comme une commande (identifié par le caractère `x`)

Si le fichier est un répertoire :

- le droit en lecture signifie que l'on peut lister ses fichiers (commande `ls`)
- le droit en écriture signifie que l'on peut y créer ou supprimer des fichiers
- le droit en exécution signifie que l'on peut le prendre comme répertoire courant ou bien le traverser pour accéder à sa sous-arborescence.

Ainsi, pour un fichier donné il existe pour chaque catégorie d'utilisateur le droit ou le non-droit d'effectuer chaque type d'opération. Il y a donc 9 droits d'accès à positionner pour chaque fichier. Ces droits d'accès se représentent par trois triplets (un triplet représente les droits `r`, `w` et `x`) de bits valant 0 ou 1 indiquant si le droit en question est autorisé ou pas. Le premier triplet correspond aux droits du propriétaire, le second triplet correspond aux utilisateurs du groupe et le troisième triplet correspond aux droits des autres utilisateurs.

Par exemple si un fichier a les droits d'accès suivant `rwxr-xr-` alors :

- le propriétaire possède les droits en lecture, écriture et exécution
- les utilisateurs appartenant au groupe du propriétaire possèdent les droits en lecture et exécution
- les autres utilisateurs possèdent uniquement le droit en lecture.

Pour afficher les droits des fichiers du répertoire courant, il est nécessaire de taper la commande `ls -l`. Seul le propriétaire du fichier et le super-utilisateur peuvent changer ses droits d'accès.

Pour changer les droits d'accès il faut utiliser la commande ***chmod***. Il existe deux manières d'exprimer des modifications de droit

- Soit en octal où chaque triplet de bits est représenté par un chiffre entre 0 et 7 :
  - $(0)_8 \rightarrow (000)_2 = - - -$: aucun droit
  - $(1)_8 \rightarrow (001)_2 = - - x$: exécution
  - $(2)_8 \rightarrow (010)_2 = -w-$: écriture
  - $(3)_8 \rightarrow (011)_2 = -wx$: écriture + exécution

◇ $(4)_8 \rightarrow (100)_2 = r--$: lecture
◇ $(5)_8 \rightarrow (101)_2 = r-x$: lecture + exécution
◇ $(6)_8 \rightarrow (110)_2 = rw-$: lecture + écriture
◇ $(7)_8 \rightarrow (111)_2 = rwx$: tous les droits

Par exemple pour ajouter tous les droits à un fichier `foo`, il faut taper la commande : `chmod 777 foo`

- soit avec un format prédéfini comportant 3 informations :
  ◇ le ou les triplets concerné(s) : `a` pour tous les triplets, `u` pour le triplet propriétaire, `g` pour le triplet groupe, et `o` pour les autres
  ◇ si il s'agit d'un ajout (caractère `+`) ou d'un retrait (caractère `-`) de droits
  ◇ les droits que l'on veut modifier : `r` pour la lecture, `w` pour l'écriture, `x` pour l'exécution

  Par exemple pour ajouter les droits en lecture et en exécution au propriétaire et au groupe d'un fichier `foo` il faut taper la commande `chmod ug+rx foo`

Chapitre 4

Les processus

4.1 Du programme vers le processus

Le développement logiciel est une production de fichiers sources écrits dans un langage de programmation (C, Java, Python, Scala, etc.). Pour qu'un programme soit compréhensible par la machine, en fonction du langage de programmation source il faut :

- soit le compiler (le traduire) en instructions binaires à l'aide d'un programme que l'on appelle **compilateur** (cas par exemple pour C/C++) et ensuite exécuter le fichier binaire.
- soit l'interpréter directement à partir du fichier source à l'aide d'un programme appelé **interpréteur**. C'est ce qui est utilisé dans le langage de script comme Python, Perl, Ruby, bash, etc.

Dans les deux cas, un programme est un ensemble d'instructions et peut être vu comme un objet inerte et statique. Contrairement à cette vision, l'exécution d'un programme apporte un aspect plus dynamique car il y a une notion d'état d'avancement du programme qui évolue au cours du temps. C'est la notion fondamentale de **processus**. Un processus n'est pas uniquement un programme en cours d'exécution mais c'est également un contexte d'exécution qui englobe :

- l'espace mémoire RAM manipulé pour pouvoir fonctionner. Cette espace mémoire regroupe aussi bien les données utilisées que les instructions du programme nécessaires à son exécution.
- un ensemble d'informations maintenues par le système d'exploitation comme :
  - ◇ le PID : un identifiant attribué par le système au moment de la création du processus
  - ◇ l'identifiant du processus parent (le PPID)
  - ◇ le nom du propriétaire, c'est à dire l'identifiant de l'utilisateur qui a lancé le processus.
  - ◇ le groupe du propriétaire (à l'instar des fichiers)
  - ◇ le cas échéant, l'identifiant de son terminal de rattachement. Le processus lancés graphiquement ne sont rattachés à aucun terminal.
  - ◇ des attributs comme :
    - · sa priorité
    - · son répertoire courant
    - · son état
    - · ses signaux reçus
    - · ses ouvertures de fichier (I/O)
    - · ...

4.2 Partage des ressources

Une machine possède une quantité de ressources matérielles bien souvent très inférieure au nombre de processus existants. On peut en effet remarquer dans les processeurs contemporains des ordinateurs grand public qu'il est tout à fait possible d'exécuter plusieurs applications en même temps (une application peut impliquer plusieurs processus) bien que ce nombre d'applications soit supérieur au nombre de cœurs de calcul. Par conséquent, nous considérerons par la suite pour simplifier que le processeur ne se résume qu'à un seul cœur de calcul. Ceci ne change en rien le problème initial qui est d'avoir plus de processus que de cœurs de calcul. Sur le même principe, on peut remarquer que la mémoire vive est une entité partagée entre les différents processus d'une machine. Cette section va décrire brièvement les mécanismes du système d'exploitation pour que ces ressources soient partagées équitablement entre les différents processus.

4.2.1 Partage du processeur

Le processeur est le composant permettant d'exécuter les instructions des programmes. Afin de permettre que chaque processus puisse évoluer dans son exécution, le système d'exploitation doit garantir :

- que le processeur soit accessible dans un temps fini
- qu'aucun processus ne monopolise son utilisation

Pour cela, les systèmes d'exploitation modernes implantent **un mécanisme de temps partagé**. Le mécanisme de temps partagé permet d'allouer une période de temps fini à chaque processus existant qui est en capacité d'évoluer dans l'exécution de son code (nous verrons plus tard qu'un processus peut être en incapacité d'évoluer pour attendre la disponibilité d'autres ressources que le processeur). Au bout d'une certaine période de temps appelée `quantum`, le système d'exploitation est chargé d'interrompre le processus en cours d'exécution, de choisir un autre processus éligible au processeur (bien souvent le processus ayant la plus haute priorité) et de poursuivre l'exécution du processus nouvellement élu. Le processus évincé du processeur sera réélu ultérieurement par le système d'exploitation. Ce mécanisme permettant de changer de processus s'appelle le mécanisme de **commutation**. La politique de choix du nouveau processus s'appelle la politique **d'ordonnancement** et peut être paramétrable en fonction des besoins d'utilisation matériel ou logiciel. Il est donc important de comprendre qu'un processus puisse être interrompu par le système d'exploitation à n'importe quel moment de son exécution pour donner lieu à une commutation. Par conséquent, il est très fréquent que le processeur ait exécuté du code d'un autre processus dans l'exécution de deux instructions consécutives d'un même programme bien que l'on est l'illusion d'un point de vu utilisateur que chaque processus soit le seul à s'exécuter sur la machine.

4.2.2 Partage de la mémoire

La mémoire est le composant matériel permettant de stocker les données de travail des processus notamment les variables déclarées dans le programme et le programme lui même. La mémoire peut être vue comme un entrepôt d'octets que l'on peut accéder en lecture ou en écriture et où chaque octet est identifié par une adresse. Les données mémoire nécessaires à l'exécution d'un processus sont partitionnées en différents segments mémoire contigus. On alloue donc pour chaque processus les segments suivants :

- **la pile d'exécution** qui regroupe toutes les variables locales ainsi que le cheminement des différents appels de fonction avec leurs arguments
- **le tas** qui regroupe tous les objets alloués dynamiquement (`malloc` pour le langage C ou `new` pour le langage Java)

- **les data** qui regroupe toutes les variables globales du processus, i.e toute variable déclarée en dehors d'une fonction
- **le text** qui regroupe les instructions binaires du processus (= le code)

Étant donné que tous les processus partagent le même composant physique, à savoir les barrettes de mémoire RAM, il est nécessaire d'avoir un mécanisme d'isolation afin qu'un processus x ne puisse pas accéder aux données manipulées par un processus y. Le matériel offre un mécanisme permettant à chaque processus de s'abstraire de l'adressage physique de chaque donnée. Ce mécanisme s'appelle la **virtualisation de la mémoire** et permet à chaque processus de posséder son propre espace d'adressage qu'on appelle alors l'espace d'**adressage virtuel** du processus. Ainsi chaque processus manipule ses données dans son propre espace d'adressage virtuel ce qui apporte les avantages suivants :

- il n'est pas possible d'accéder à l'espace d'adressage d'un autre processus ce qui apporte une bonne isolation. On peut de ce fait considérer qu'un processus pense être le seul à s'exécuter sur la machine de son point de vue.
- l'adressage virtuel est complètement décorréler de l'adressage physique ce qui permet donc de s'abstraire de l'emplacement physique des données de chaque processus et de la taille réelle de la mémoire.

Par conséquent le processeur manipule non pas des adresses physiques mais les adresses virtuelles du processus qui est en cours d'exécution. Pour pouvoir traduire une adresse virtuelle d'un processus vers une adresse physique du composant mémoire, le matériel offre un composant appelé Memory Management Unit (MMU). Ainsi, lorsque le processeur souhaite accéder à une adresse mémoire (virtuelle), cette adresse est traduite automatiquement et matériellement en adresse physique par la MMU. On notera donc qu'il faut maintenir pour chaque processus une table de traduction permettant de faire la correspondance entre l'adressage virtuel et l'adressage physique. Cette table de correspondance s'appelle la **table des pages**.

4.3 Organisation des processus

Dans un système d'exploitation Unix, les processus sont organisés logiquement de manière hiérarchique. Il y a donc une notion de filiation entre les processus. Nous verrons plus loin dans ce document la nature du lien qui relie un processus à ses processus fils. Il vous est possible d'avoir un visuel sur cette hiérarchie en tapant la commande `pstree` qui affichera sous forme d'arbre l'ensemble des processus existants dans le système.

4.4 États des processus

Les processus suivent la machine à états de la figure 4.1 au cours de leur existence.
On distingue 4 états :

- l'état **prêt** : le processus est en attente du processeur. C'est l'état de départ de tout processus et l'état incontournable pour obtenir le processeur. Le processus sort de cet état uniquement pour passer à l'état *élu* quand le système d'exploitation le choisi lors d'une commutation.
- l'état **élu** : le processus utilise le processeur est en train de s'exécuter. On sort de cet état pour trois raisons :
  - ◇ le système d'exploitation décide d'évincer le processus courant car il est arrivé en fin de quantum et repasse à l'état *prêt*
  - ◇ le processus doit attendre un événement particulier et passe à l'état *en attente*.
  - ◇ le processus n'a plus d'instruction à exécuter (exit)

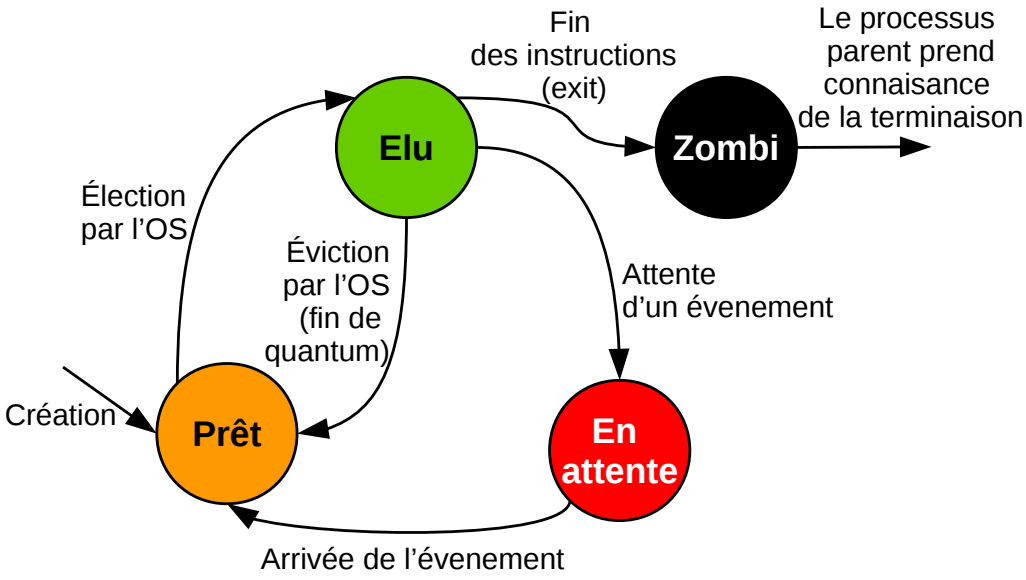


FIGURE 4.1 – Machine à états d'un processus Unix

On notera que toute sortie de l'état *élu* donnera forcément lieu à une nouvelle élection si il existe des processus à l'état *prêt*.

- l'état **en attente** : le processus attend un événement particulier autre que l'obtention du processeur (ex : le contenu d'un fichier suite à une requête d'entrée/sortie sur un périphérique). On ne peut sortir de cet état que lorsque l'événement attendu est arrivé.
- l'état **zombi** : le processus a terminé son exécution et attend que son parent prenne connaissance de sa terminaison. Cet état sera développé dans la section traitant de la terminaison d'un processus (section 4.8).

Il est possible de consulter l'état des processus sur la première lettre (majuscule) du champs de colonne STAT de la commande `ps aux`. On pourra remarquer que du point de vue utilisateur, les états *élu* et *prêt* sont confondus et portent tous les deux la lettre **R** (Running/Runnable). En effet, la distinction n'est utile que pour le système d'exploitation. L'état *en attente* est subdivisé en deux états : les lettres **S** et **T**. L'état **T** permet d'indiquer que l'attente ne se débloquera qu'à la réception d'un signal SIGCONT (voir section 4.9.1). L'état zombi est représenté par la lettre **Z**.

4.5 Interruption, mode utilisateur et mode système

Le système d'exploitation est souvent vu comme une entité suprême et indépendante des processus. Il n'en n'est rien. En réalité, puisqu'il fait l'interface entre le logiciel applicatif et le matériel de la machine il peut être vu comme un gestionnaire d'interruptions. Une interruption est un événement qui nécessite un traitement de la part du système d'exploitation. L'événement peut être :

- soit d'origine matérielle comme par exemple :
  - ◇ une interruption horloge qui permet de mettre à jour régulièrement les données relatives au temps et de pouvoir ainsi détecter les fins de quantum
  - ◇ une notification de fin d'entrée/sortie pour notifier que des données rapatriées depuis un périphérique (ex : disque) sont prêtes à être utilisées en mémoire.
- soit d'origine logicielle qui sont typiquement les appels systèmes. Un appel système est une fonctionnalité offerte par le système d'exploitation aux applications comme par exemple créer ou tuer un processus, créer/lire/écrire un fichier, etc...

Ainsi, si nous zoomons sur l'état *élu* (figure 4.2), on peut remarquer qu'un processus en cours d'exécution ne cesse d'alterner entre deux sous-états appelés mode utilisateur (abrévié "mode U") et mode système (abrévié "mode S").

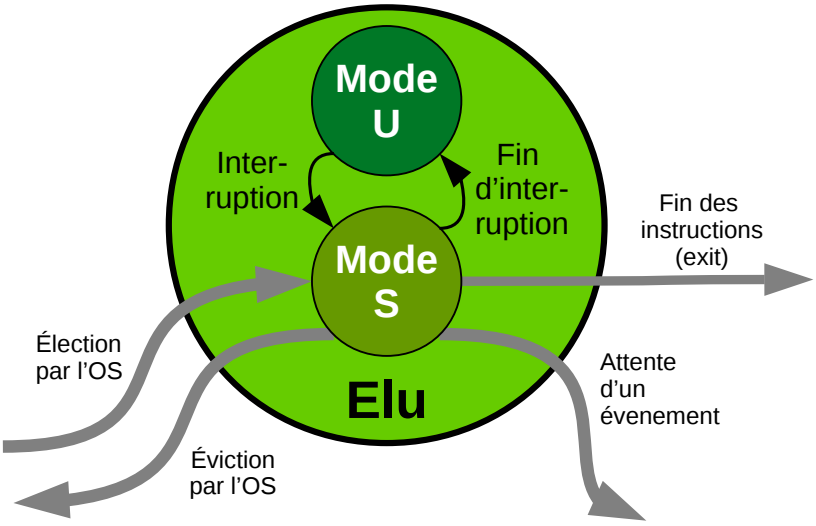


FIGURE 4.2 – Sous-états de l'état *élu*

Le mode U correspond au mode où le processus exécute le code du programme défini par l'utilisateur. Toute application du système s'exécute donc en mode utilisateur. Lorsque une interruption survient, le processus bascule en mode S et exécute le code du système d'exploitation permettant de traiter l'interruption. En mode S l'espace d'adressage du processus est enrichi par les segments mémoires propres au système d'exploitation ce qui permet notamment de mettre à jour les données du système. En mode U les segments systèmes sont inaccessibles évitant ainsi à tout code utilisateur d'accéder aux variables système ce qui pourrait constituer une faille de sécurité.

Bien que le mode S soit le seul état de sortie du mode U, on peut remarquer que ceci n'est pas réciproque. En effet, en mode S le processus peut très bien être commuté et repasser à l'état *prêt* ou bien se mettre en attente laissant ainsi la place à un autre processus. On remarquera également que le mode S est un passage obligatoire pour être à l'état *élu*.

4.6 Les entrées/sorties

On parle d'entrée/sortie (ou I/O) lorsqu'un processus échange des données avec un périphérique de la machine (disque, carte réseau, souris, carte graphique, ...).

Bien que les I/O soient très coûteuses par rapport aux accès mémoire, elles sont incontournables dans la vie d'un processus. Comme tout périphérique est représenté par un fichier sous Unix (voir section 3.3), l'envoi et la réception des données se font de manière unifiée quelque soit le périphérique. La communication avec un périphérique peut se voir comme un **flux de données** :

- que l'on ouvre à sa création (c'est à ce moment que l'on indique si on souhaite ouvrir en mode lecture seule, en mode écriture seule ou les deux)
- que l'on active dans un sens ou dans un autre en fonction de ce que l'on veut faire (recevoir ou envoyer des données, i.e lire ou écrire)
- que l'on ferme lorsque l'on n'en a plus l'utilité ou bien lorsque le processus se termine

Lors de l'exécution d'un processus, trois flux sont automatiquement ouverts et disponibles :

- un flux ouvert en lecture appelé **entrée standard** dont le périphérique par défaut est le clavier (identifié par l'ouverture numéro 0)
- deux flux ouverts en écriture appelés **sortie standard** et **sortie erreur** (identifiées respectivement par l'ouverture numéro 1 et 2) dont le périphérique par défaut est le terminal dans lequel le processus s'exécute. En d'autres termes, c'est à partir de ces deux flux que la commande produit ses affichages.

Pour illustrer ceci, on peut utiliser la commande ***cat*** qui permet d'afficher le contenu des fichiers passés en argument. Si cette commande est lancée sans argument, elle lit tout donnée lue sur son entrée standard pour l'écrire sur sa sortie standard.

```
bash:~$ cat
bonjour
bonjour
au revoir
au revoir
```

Dans cet exemple, une fois qu'un processus exécute la commande cat sans argument, toute ligne tapée au clavier est affichée immédiatement.

Toute ouverture de flux d'I/O est caractérisée par un compteur indiquant un déplacement (ou offset) dans le flux. Le déplacement indique l'identifiant du prochain octet à accéder dans le flux et est mis à jour à chaque nouvel accès. Ceci peut être vu comme un curseur que l'on déplace à chaque accès. Ainsi si nous souhaitons lire n octets sur le flux, le compteur d'offset sera incrémenté de n.

Il est bien sûr possible de changer le périphérique sous-jacent de ces trois flux en redirigeant ces derniers sur un autre périphérique. La syntaxe :

- pour une redirection de l'entrée standard :
  $commande\ [argument_1\ ...\ argument_n] < fichier$
  Si $fichier$ représente un fichier régulier alors le programme ne prendra plus le clavier en tant que entrée standard mais le contenu du fichier
- pour une redirection de la sortie standard :
  - ◇ si on souhaite écrire au début du flux (offset = 0) :
    $commande\ [argument_1\ ...\ argument_n] > fichier$
    Dans le cas d'un fichier régulier, ceci aura pour conséquence d'écraser son contenu
  - ◇ si on souhaite écrire à la fin du flux (offset= fin du fichiers) :
    $commande\ [argument_1\ ...\ argument_n] >> fichier$
    Dans le cas d'un fichier régulier, ceci aura pour conséquence d'ajouter à la fin du fichier ce que la commande produit sur sa sortie standard
- pour une redirection de la sortie erreur : idem que la sortie standard sauf qu'il faut remplacer $>$ et $>>$ par $2>$ et $2>>$.

On notera en shell que toute redirection de flux implique une nouvelle ouverture et donc un nouvel offset.

4.7 Création de processus

La création d'un nouveau processus se fait par le système à la demande de l'utilisateur et se fait toujours à partir d'un processus existant appelé **processus parent**. Comme nous l'avons vu en section 4.3, à l'instar des fichiers, les processus sont aussi organisées de manière hiérarchique. D'un point de vue du shell, un nouveau processus est créé automatiquement lorsque l'on entre une commande dans le terminal (excepté pour quelques commandes internes au shell comme par exemple

echo ou *cd*, voir section 6.2). Une fois la commande lancée, le processus nouvellement créé exécute le code de la commande. Son processus parent est le processus de l'interpréteur de commande (i.e. le shell).

Sans trop rentrer dans les détails, pour créer un processus fils, un processus parent doit dans un premier temps dupliquer toute sa mémoire et ensuite modifier les différentes variables du systèmes propre au fils comme par exemple le PID ou sa table des pages. On peut donc noter qu'une fois le fils créé, beaucoup de données sont hérités du père notamment tous ses segments et l'ensemble des ouvertures de fichier si celle-ci n'ont pas été redéfinies. On notera que dans la grande majorité des cas, un fils n'exécute pas le même code que son père. Par conséquent, il est donc nécessaire de changer les valeurs des segments mémoires du fils. Ce changement n'est pas propre à la création de processus et doit se faire explicitement par le nouveau processus (à travers l'appel système `exec`) lorsqu'il sera élu pour la première fois.

4.8 Terminaison d'un processus

Un processus se termine lorsqu'il n'a plus de code utilisateur à exécuter ou bien lorsqu'il rencontre une instruction de terminaison. À sa terminaison, un processus possède une valeur appelée **code de retour** à laquelle son processus parent peut y accéder. Conventionnellement un code de retour nul correspond à un comportement normal du processus (pas d'erreur) alors qu'un code supérieur à zero désigne une situation anormale. Un processus se termine lorsqu'il exécute la commande ***exit***. Cette commande peut prendre un argument qui est le code de retour du processus.

Nous avons vu dans la machine à états d'un processus (voir section 4.4) que la sortie de l'état *élu* en cas de terminaison aboutissait dans l'état *zombi*. L'état *zombi* est un état transitoire avant la mort définitive du processus. Dans cet état, le processus a libéré son espace mémoire mais le système maintient tout de même un minimum d'information sur le processus (notamment son code de retour) jusqu'à ce que le père en prenne explicitement connaissance. En shell, pour prendre connaissance de la terminaison de **tous ses fils**, un processus doit appeler la commande ***wait***. Cette commande est bloquante tant qu'il existe un fils qui ne soit pas à l'état zombi ce qui implique que le processus père passera à l'état *en attente* jusqu'à ce que cette condition soit remplie. Si en revanche tous les fils sont zombis, alors la commande *wait* n'est pas bloquante. Dans les deux cas le système assure qu'après la commande *wait* tous les processus fils aient été détruits définitivement.

Le cas où le processus parent se termine et est détruit avant ses fils entraîne des processus orphelins. Pour assurer que tout processus ait un parent, le système fait adopter tout processus orphelin par le processus init qui a le pid numéro 1. Lorsque le système a démarré, le rôle du processus init est donc d'adopter tous les processus orphelins et d'appeler en perpétuellement *wait* pour débloquer les fils zombis.

4.9 Communication inter-processus

Nous avons pu remarquer que la communication directe entre processus via la mémoire était impossible étant donné que cette dernière est virtualisée pour fournir une isolation entre les processus. Le système propose cependant différents mécanismes plus ou moins évolués pour que cette communication soit tout de même possible.

4.9.1 Les signaux

Le signal est un moyen de communication très rudimentaire. Il permet à tout processus de pouvoir envoyer une notification à un processus destination. Cette notification se résume par le passage d'un

bit de 0 à 1. Ainsi le système maintient pour tout processus un vecteur de bit où chaque bit correspond à un signal particulier. Dans les architectures actuelles, le nombre de signaux dans le système est égal à 64. Chaque signal est ainsi identifié par un numéro compris entre 1 et 64. Pour les identifier plus facilement d'un point de vue humain, chaque signal possède également un nom. Il est possible d'avoir la liste des signaux du système avec la commande ***kill -l*** qui donne l'affichage suivant :

```
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

Envoi d'un signal

Pour envoyer un signal il faut utiliser la commande ***kill***. La commande ***kill*** nécessite deux arguments :

- le premier argument pour le numéro ou le nom du signal à envoyer
- le deuxième argument pour le pid du processus destinataire

Réception d'un signal

Lorsqu'un signal est reçu, le bit correspondant sur le processus destinataire prend la valeur 1. Lorsque le processus destinataire sera élu, il prendra connaissance des signaux reçus (i.e. tous les bit égaux à 1) avant de rentrer en mode U. Tout signal pris en compte remettra le bit correspondant à 0. Il faut donc noter que le fait que le bit d'un signal soit égal à 1 implique que le signal ait été reçu au moins une fois. En effet comme les signaux reçus ne sont stockés dans une file d'attente, il est donc impossible de différencier si le signal a été envoyé une fois ou plusieurs fois avant de reprendre la main sur le processeur.

Traitement d'un signal

La réception d'un signal sur un processus engendre un traitement particulier. Ce traitement est une portion de code qui est exécutée en mode U. Une fois ce traitement effectué, le processus continue son exécution à l'endroit de son code utilisateur où il a été interrompu. Tous les signaux ont un traitement par défaut qui pour la grande majorité d'entre eux se résume à exécuter la commande `exit` et donc de tuer le processus. On notera que le signal SIGSTOP permet de mettre en attente le processus destinataire (état "*en attente*") jusqu'à ce qu'il reçoive le signal SIGCONT (qui le fera passer à l'état "*prêt*").

Cependant, il est possible de redéfinir le comportement des traitements de signaux avec la commande ***trap***. La commande `trap` prend 2 arguments :

- le premier argument est une commande ou une liste de commandes que le processus exécutera à la réception du signal
- le deuxième argument est l'identifiant du signal à redéfinir

Par exemple si nous voulons faire afficher le mot "bonjour" à la réception d'un SIGINT il faut taper la commande :

```
trap "echo bonjour" SIGINT
```

Pour revenir au comportement par défaut le premier argument de `trap` doit être égal à `-`. Le système maintient donc pour chaque processus une table qui associe un type de signal à une routine de traitement. Les signaux SIGKILL et SIGSTOP permettent respectivement de tuer et d'arrêter un processus et ne peuvent pas être redéfinis afin d'avoir toujours la possibilité d'interrompre un processus depuis l'extérieur. Ceci est utile par exemple lors de l'extinction de l'ordinateur.

Limites des signaux

Lorsque l'on utilise des signaux il est impossible :
- de connaître l'émetteur
- de véhiculer un message autre que le type du signal
- de savoir quand le signal sera traité
- de savoir si le signal a déjà été traité
- de connaître le nombre d'instances reçus d'un même signal

4.9.2 Les fichiers

Le système de fichiers constitue un espace d'adressage commun à tous les processus du système. Il est donc possible de faire communiquer deux processus à travers la lecture et l'écriture de fichiers. Bien que cette communication soit loin d'être optimale en termes de performances à cause des I/O qu'elle peut engendrer, c'est une solution relativement simple à mettre en œuvre et suffisante en shell. Il faut néanmoins prendre en compte que l'opération d'accès à un fichier n'est pas atomique et peut donc être interruptible pour commuter vers un autre processus qui utilise le même fichier. Des mécanismes de synchronisation sont donc à utiliser pour éviter les incohérences.

4.9.3 Les tubes

Il est possible de chaîner plusieurs processus en pipeline grâce à un mécanisme offert par UNIX : le tube. Ce chaînage de processus consiste à faire en sorte que la sortie standard d'un processus alimente directement l'entrée standard du suivant. Le symbole utilisé pour créer des tubes est la barre verticale |. Un chaînage de processus par les tubes s'écrit donc

```
cmd1 | cmd2 | ..... | cmdN
```

Ce genre de pratique est très utilisé pour extraire par exemple un contenu particulier d'un fichier de log qui comporte beaucoup d'informations. Les commandes les plus utilisées dans le cas des tubes sont les commandes suivantes :
- *sort* : permet de trier l'entrée standard par défaut selon l'ordre lexicographique
- *uniq* : permet de supprimer les lignes doublons consécutives
- *grep* : permet de sélectionner des lignes selon un schéma décrivant une expression rationnelle
- *cut* : sélectionne un ensemble de caractères sur une ligne
- *tr* : permet principalement de remplacer un caractère par un autre
- *sed* : éditeur de flux plus élaboré (modifier des lignes, supprimer des lignes, ...)
- *head* : sélectionne les n premières lignes d'un flux
- *tail* : sélectionne les n dernières lignes d'un flux

Voici quelques exemples de commandes avec les tubes :
- Afficher uniquement le nom des répertoires présents à la racine du répertoire personnel
  `ls -la ~ | grep -e "^d" | tr -s ' ' | cut -d ' ' -f 9`
- Afficher en ligne uniquement les PID des processus existant de la machine
  `ps aux |tr -s ' '|cut -d ' ' -f2 | tr '\n' ' '`

- Afficher de manière croissante, 20 nombres tirés aléatoirement
  `rand -N 100 | tr ' ' '\n' | sort -n`

4.9.4 Autres solutions existantes

Les moyens de communication inter-processus suivants ne sont pas disponibles en shell mais dans des API plus bas niveau comme le langage C. Elles ne seront pas développées dans le cadre de ce document.

Les segments de mémoire partagée

Il est possible d'allouer dans l'espace d'adressage d'un processus un segment de mémoire qui peut être partagé avec d'autres processus. Concrètement, pour avoir un segment de mémoire partagée entre plusieurs processus, le système doit faire correspondre pour chaque page virtuelle de chaque processus du segment partagé la même zone physique dans la mémoire RAM. Un gros avantage de la mémoire partagée c'est qu'elle évite les I/O et permet donc une communication plus directe et plus efficace mais nécessite dans la plupart des cas la mise en place de mécanismes de synchronisation.

Les files de messages

Dans cette solution, les processus communiquent par l'intermédiaire d'une entité logicielle faisant office de "boite à lettres" dans laquelle il est possible de déposer ou de retirer un ou plusieurs messages. La file de messages permet de découpler temporellement les processus communicants (communication asynchrone) et évite de mettre en place des mécanismes de synchronisation.

Les sockets

Les sockets est une API permettant de faire communiquer deux processus qui peuvent être sur deux machines distantes à travers le réseau.

Chapitre 5

Shell-script

Ce chapitre décrit les différents outils pour pouvoir écrire un programme en shell. Ce type de programme est appelé shell-script. En pratique, il est principalement utilisé dans les phases d'administration du système permettant ainsi d'automatiser une séquence de commandes. Ces commandes constituent un scénario d'actions. Un shell-script est concrètement un fichier texte que l'on pourra exécuter, c'est à dire, le considérer comme une commande à part entière.

5.1 Le shebang

Dans la pratique, on utilise plusieurs invites de commandes pour commander des systèmes ou des applications différentes. Par exemple, il est possible d'utiliser les interpréteurs Perl, Python, Bash (Bourne Again Shell), Tcsh. Il ne suffit donc pas d'écrire une suite d'instructions pour que le système puisse l'exécuter. Il faut également préciser l'interprèteur pour lequel ce script est écrit. C'est l'objet de la première ligne d'un script : la ligne "shebang". Elle commence par un dièse et un point d'exclamation suivi du chemin absolu de binaire de l'interpréteur. En bash nous écrirons donc :

```
#!/bin/bash
```

5.2 Commentaire

Les commentaires sont des lignes qui seront ignorées par l'interpréteur. En shell, les commentaires sont tous les caractères compris entre un # et un retour chariot (passage à la ligne).

5.3 Les variables

5.3.1 Lecture et écriture

Comme dans tout langage de programmation, il est souvent indispensable de manipuler des variables. En bash les variables ne sont pas typées et leur contenu est considéré comme du texte. La déclaration d'une variable se fait à la première affectation. Pour affecter une valeur dans une variable, il faut écrire le nom de la variable suivi d'un = puis de la valeur à écrire. Pour lire la valeur d'une variable il faut impérativement ajouter un $ devant son nom. Par exemple le script :

```
#!/bin/bash
foo="Bonjour"
echo $foo
```

affiche le contenu de la variable foo qui est "bonjour".

5.3.2 Les variables spéciales

Dans un script, il existe déjà des variables prédéfinies qui comportent des informations sur le contexte d'exécution. On peut noter les variables comme :

- `$@` : contient l'ensemble des arguments de la commande
- `$#` : contient le nombre d'arguments de la commande
- `$1`, `$2` ... : contient le premier, deuxième .... argument de la commande
- `$$` : contient le PID du processus qui exécute le script
- `$?` : contient le code de retour de la dernière commande lancée

5.3.3 Les variables d'environnement

Les variables d'environnement sont un ensemble de variables dont les valeurs sont transmises de processus père en processus fils. Ainsi toute variable présente dans l'environnement du processus parent sera copiée dans l'environnement de chaque processus créé.

Pour déclarer une variable dans l'environnement il faut utiliser la commande ***export*** suivi du nom de la variable concernée.

Par exemple : si nous considérons deux scripts *pere.sh* et *fils.sh* :

```
#!/bin/bash
#pere.sh
var=bonjour
./fils.sh
export var
./fils.sh
echo $var
```

```
#!/bin/bash
#fils.sh

echo "var = $var"
var="au revoir"
```

Ceci affiche :

```
var=
var=bonjour
bonjour
```

On notera que la transmission des variables d'environnement se fait par copie ce qui implique que toute modification dans un processus n'aura aucune incidence sur les copies des autres processus.

Au lancement d'un script, il existe déjà un ensemble de variables d'environnement pré-définies qu'il est possible d'afficher avec la commande ***env***.

5.4 Les tests

Les tests (ex : de comparaisons) se font avec la commande *test* ou bien avec la commande [. Si le test est vrai alors le code de retour de la commande vaut 0. Sinon si le test est faux alors son code de retour est différent de 0.

Attention : contrairement à la majorité des langages, la valeur 0 désigne le booléen vrai en bash. La commande `test` permet de :

- comparer numériquement deux opérandes. Les opérandes doivent être interprétables en tant que entier, c'est à dire qu'elles ne doivent comporter que des caractères numériques :
  - ◇ `[ operande1 -eq operande2 ]` pour tester l'égalité numérique
  - ◇ `[ operande1 -ne operande2 ]` pour tester l'inégalité numérique

  - ◇ `[ operande1 -lt operande2 ]` pour tester si `operande1` est strictement inférieure à `operande2`
  - ◇ `[ operande1 -le operande2 ]` pour tester si `operande1` est inférieure ou égale à `operande2`
  - ◇ `[ operande1 -gt operande2 ]` pour tester si `operande1` est strictement supérieure à `operande2`
  - ◇ `[ operande1 -ge operande2 ]` pour tester si `operande1` est supérieure ou égale à `operande2`
- comparer la valeur de deux opérandes quelconques
  - ◇ `[ operande1 = operande2 ]` pour tester l'égalité
  - ◇ `[ operande1 != operande2 ]` pour tester l'inégalité
- faire des tests sur l'arborescence de fichiers :
  - ◇ `[ -e fich ]` pour tester si un fichier existe
  - ◇ `[ -d fich ]` pour tester si le fichier existe et si c'est un répertoire
  - ◇ `[ -f fich ]` pour tester si le fichier existe et si c'est un fichier classique

5.5 Les quotes

On note trois types de quote :

- les simples quotes : délimitent une chaîne de caractères. Même si cette chaîne contient des commandes ou des variables shell, celles-ci ne seront pas interprétées. Exemple :
  ```
  variable="secret"
  echo 'Mon mot de passe est $variable.'
  ```
  affiche `"Mon mot de passe est $variable."`
- les doubles quotes : délimitent une chaîne de caractères, mais les noms de variable sont interprétés par le shell. Par exemple :
  ```
  variable="secret"
  echo "Mon mot de passe est $variable."
  ```
  affiche `"Mon mot de passe est secret."`. Ceci est utile pour générer des messages dynamiques au sein d'un script.
- Back-quote (ou accent grave) : le shell considère que les Back-quotes délimitent une commande à exécuter. Les noms de variable et les commandes sont donc interprétés et la sortie standard de la commande est stockable dans une variable. Par exemple :
  ```
  foo=1
  variable=`expr 5 + $foo`
  echo "$variable"
  ```
  affiche `6`. Les back-quotes peuvent également être remplacés sous la forme de variable par `$(...)`. Ainsi écrire `` `cat fichier` `` est équivalent à écrire `$(cat $fichier)`

5.6 Expression numérique

Comme nous l'avons vu dans le section 2.3.4, la commande `expr` permet d'évaluer des expressions arithmétiques. Cependant pour exploiter le résultat de l'expression, il est souvent utile de le stocker dans une variable. Pour cela il faut donc entourer la commande `expr` par des back-quotes ou bien par `$(...)`. Afin de simplifier cette syntaxe, le shell offre un raccourci syntaxique en utilisant `$(( expression ))`. Cette syntaxe offre l'avantage également de rendre facultative l'utilisation des dollars lors de la lecture des variable

Si on considère l'existence de deux variables `x` et `y`, les 4 expressions suivantes sont équivalentes :

```
var=`expr $x + $y`
var=$(expr $x + $y)
var=$(($x + $y))
```

```
var=$((x+y))
```

5.7 Les structures de contrôle

5.7.1 Structure conditionnel

Comme dans tout langage de programmation, les branchements conditionnels permettent d'exécuter un code ou un code alternatif en fonction de la valeur d'une expression. L'expression en shell est une commande : elle vaut vrai si la commande a un code de retour est égal à 0 et faux pour tout autre valeur. La syntaxe est la suivante :

```
if cmd      #vrai si le code de retour de cmd vaut 0
then
   # instructions si vrai
else
   # instructions si faux
fi
```

La partie `else` n'est pas obligatoire. En revanche le mot clé `fi` doit toujours être là pour marquer la fin de l'instruction conditionnelle.

Il est également possible d'avoir plusieurs alternatives avec le mot clé `elif` :

```
if cmd1      #vrai si le code de retour de cmd1 vaut 0
then
   # instructions si vrai
elif cmd2
   # instructions si cmd1 faux et cmd2 vrai
else
  #  instructions si cmd1 faux et cmd2 faux
fi
```

5.7.2 Les boucles

Comme dans tout langage de programmation, ces structures permettent de répéter une partie du code autant de fois expression soit vrai. La syntaxe de la boucle "tant que" est :

```
while cmd  #vrai si le code de retour de cmd vaut 0
do
   #instruction boucle
done
```

la syntaxe de la boucle "pour chaque"

```
for variable in 'valeur1' 'valeur2' 'valeur3'
do
        # faire le traitement pour $variable
done
```

Il est également possible de faire une boucle for plus classique, c'est à dire en décidant à l'avance du nombre d'itérations que l'on souhaite faire

- soit avec la commande `seq` :

```
for i in $(seq 1 10)
do
    echo $i
done
```

- soit avec une syntaxe plus classique des langages de programmation

```
for ((i=0 ; 10 - $i ; i++))
    do echo $i
done
```

Les débranchements `break` et `continue` sont également possibles dans une boucle.

Chapitre 6

Exécution de commande

Ce chapitre décrit les différents mécanismes pour exécuter une commande. Il nécessite d'avoir assimilé les notions du chapitre 3 sur les fichiers, du chapitre 4 sur les processus et du chapitre 5 sur le shell-script.

6.1 Fichier de la commande

Pour lancer une commande, il est d'abord nécessaire que l'interpréteur de commande connaisse le chemin du fichier qui contient le code de cette commande dans l'arborescence du système de fichier. **Ce fichier doit avoir les droits en exécution. Si le fichier est un script, les droits en lecture doivent également être positionnés.**

La variable `PATH`

On peut remarquer que la plupart du temps on entre directement le nom de la commande sans entrer de nom fichier. Ceci permet d'éviter de taper l'ensemble du chemin absolu de chaque commande que l'on souhaite lancer. Pour ce faire, le shell définit la variable d'environnement `PATH`. Cette variable contient une liste de répertoires (chaque élément de la liste est séparé par `:`). En ajoutant le chemin d'un répertoire dans cette variable, tout fichier exécutable de ce répertoire pourra être exécuté sans qu'il soit nécessaire de préciser le chemin du fichier. Autrement dit, si cette variable était vide, il serait obligatoire de préciser à chaque commande le chemin absolu du fichier associé. Par défaut le contenu de la variable `PATH` est :

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

On peut remarquer que la variable PATH contient la liste des répertoires des commandes fournies de base par le système (répertoire `/bin`) ainsi que les commandes pour lancer les logiciels fournis par la distribution ou bien ceux qui ont été installés par l'utilisateur. Ainsi lorsque l'utilisateur entre une commande, l'interpréteur suit l'algorithme :

```
si la commande est un chemin (relatif ou absolu)
   si le fichier n'existe pas ou n'a pas les droits en exécution
        échec du lancement (commande inexistante)
   sinon
        exécuter la commande
sinon
   parcourir les répertoires de la variable PATH de gauche à droite
```

```
  pour chaque fichier f du répertoire que l'on parcours
      si f a le même nom que la commande et a les bons droits
           arrêter le parcours et exécuter la commande
si pas trouvé le bon fichier f
  échec du lancement(commande inexistante)
```

Il faut noter que l'ordre des répertoires dans la variable `PATH` est important. En effet si deux fichiers exécutables ont le même nom mais dans deux répertoires différents alors c'est le premier répertoire rencontré dans le parcours de la variable `PATH` qui primera sur l'autre. Dans ce cas le seul moyen de lancer la commande qui a été masquée est de renseigner explicitement son chemin.

6.2 Processus et commande

Lors du lancement d'une commande un processus sera créé implicitement par l'interpréteur dans les cas suivants :

- la commande n'est pas un mot clé du shell (`if`, `for`, affectation de variable,...)
- la commande n'est pas une commande interne du shell (`echo` , `read`, `source`, `cd`, `export`, `pwd`, `shift`, `test`, `trap`, `kill`, `wait`, `exit`, ...)

En shell, lorsqu'un processus est créé par le lancement d'une commande, le processus parent qui a lancé la commande (le terminal ou bien un script) se met implicitement en attente de terminaison du processus fils. Il y a donc un `wait` implicite à chaque création de processus. Ceci donne ainsi l'illusion d'un appel de fonction classique au sein d'un programme à la différence que le code appelant et le code appelé sont exécutés par deux processus différents.

Exécution en arrière plan

Il est possible de lancer une commande engendrant un processus sans attendre la fin de celui-ci en ajoutant le caractère `&` à la fin. Le processus père ne se bloquant pas dans l'état "*en attente*", ceci permet ainsi d'avoir deux processus qui s'exécutent de manière concurrente.

La commande `source`

La commande `source` permet de lancer un script shell sans créer de nouveau processus. Le code du script appelé est donc ainsi exécuté dans le même processus et le même contexte que le script appelant. Par conséquent :

- toute variable déclarée dans le script est connue dans le script appelé
- toute variable déclarée dans le script appelé sera connue du script appelant une fois l'appel terminé
- toute modification dans le script appelé, d'une variable déclarée dans le script appelant sera répercutée dans le script appelant une fois l'appel terminé

Synthèse

La figure 6.1 résume et schématise les trois modes d'exécution d'une commande décrits ci-avant. Les trois schémas sont des chronogrammes où le temps défile du haut vers le bas. Chaque processus est modélisé par un fil d'exécution coloré en fonction de l'état du processus.

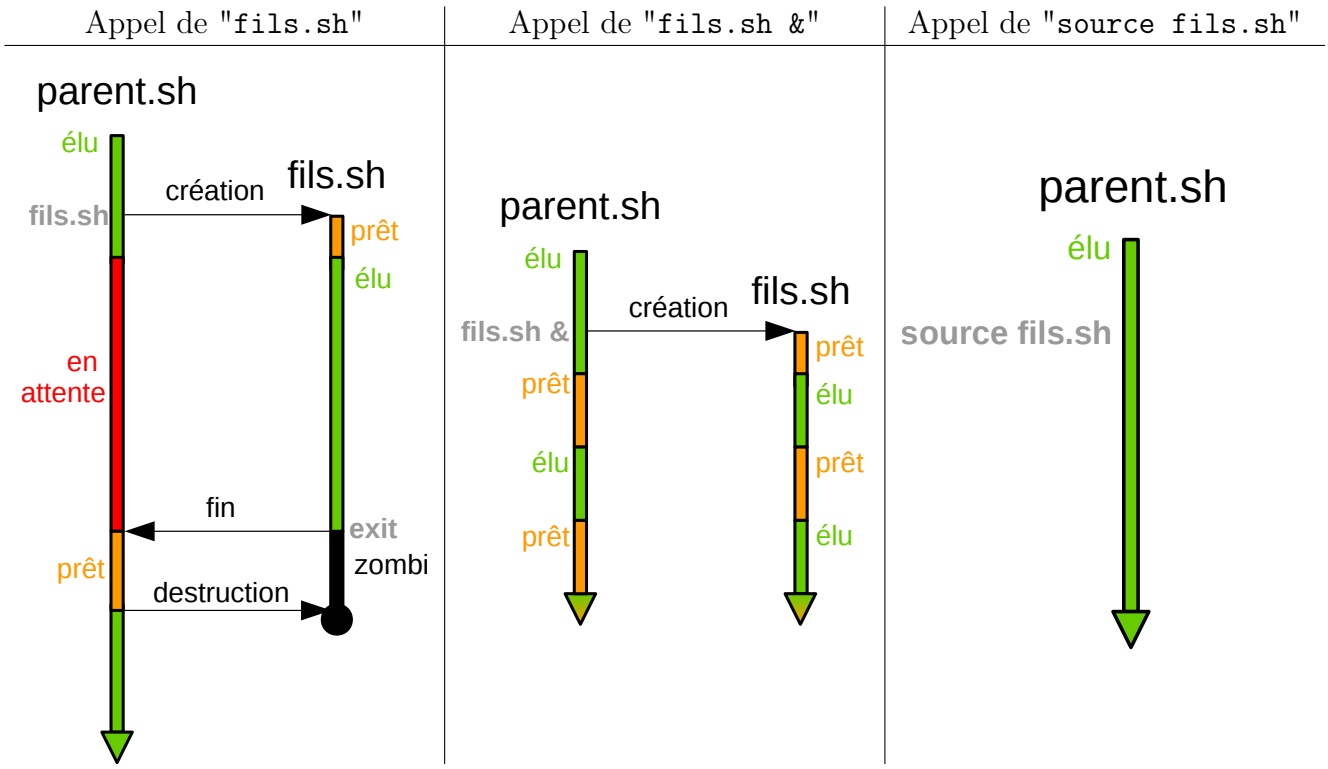


FIGURE 6.1 – Synthèse des différents modes d'exécution

Chapitre 7

Synchronisation de processus

Dans un système d'exploitation, l'exécution concurrente de processus nécessite souvent une synchronisation entre eux. On entend par synchronisation la capacité de stopper volontairement certains processus jusqu'à ce qu'une condition soit respectée. Jusqu'à présent nous avons vu la commande `wait` (cf section 4.8) qui bloque un processus tant que tous ses processus fils ne sont pas terminés. Cependant il existe d'autres mécanismes de synchronisation notamment lors d'accès à une ressource partagée (fichier, variable) afin d'éviter les incohérences sur celle-ci.

7.1 Ressources partagées et concurrence

Prenons le script suivant qui modifie une valeur entière contenue dans un fichier en l'incrémentant de 1 :

scripts/incr.sh

```
#!/bin/bash
#incr.sh

fic=$1

read val <$fic
val=$(($val + 1))
echo $val >$fic
```

Le script `incr_seq.sh` suivant initialise la valeur 0 dans un fichier, lance 3 scripts `incr.sh` puis affiche la valeur du fichier.

scripts/incr_seq.sh

```
#!/bin/bash
#incr_seq.sh

fichier=/tmp/cpt

echo 0 > $fichier

./incr.sh $fichier
./incr.sh $fichier
./incr.sh $fichier

```

```
cat $fichier
```

En exécutant ce script, on obtient sans surprise la valeur 3 dans le fichier.

```
$ ./incr_seq.sh
3
```

Voici une autre version de ce script qui lance 3 scripts mais de manière concurrente.

scripts/incr_conc.sh

```
#!/bin/bash
#incr_conc.sh

fichier=/tmp/cpt

echo 0 > $fichier

./incr.sh $fichier &
./incr.sh $fichier &
./incr.sh $fichier &

wait

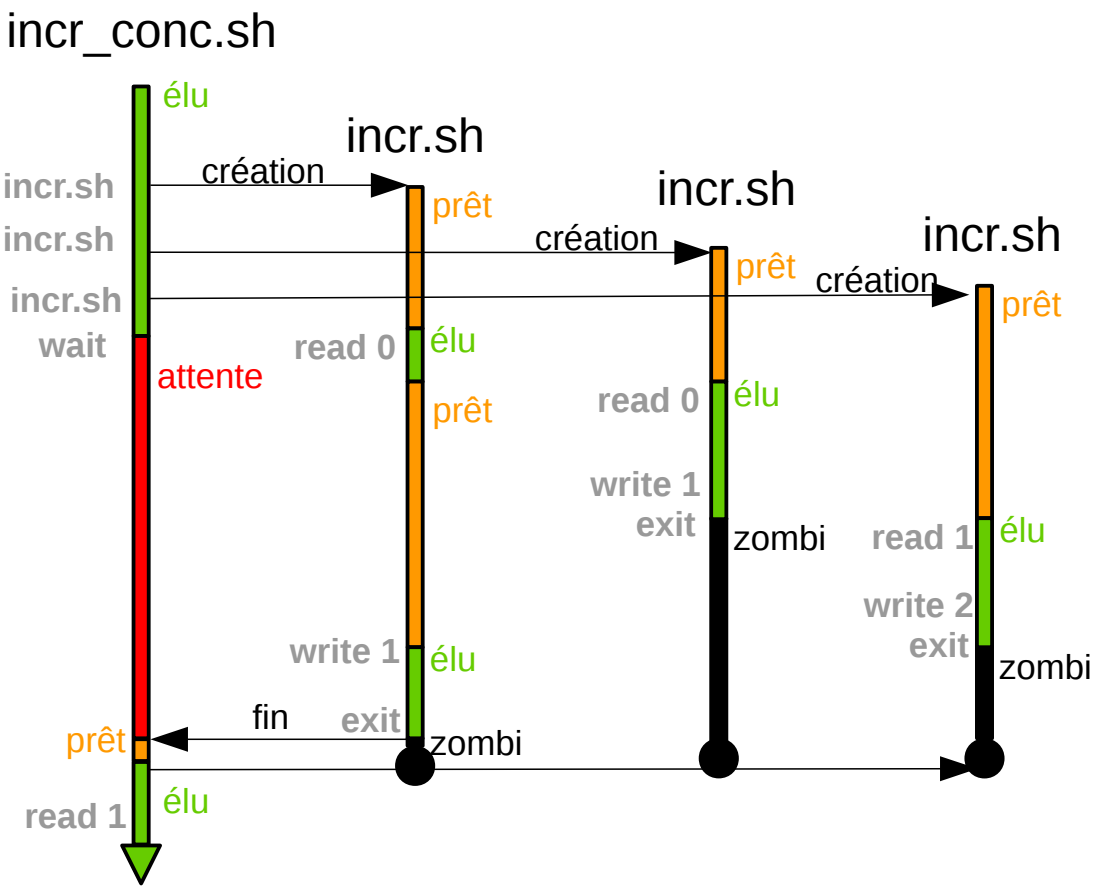
cat $fichier
```

En exécutant ce script on obtient la valeur 1 dans le fichier.

```
$ ./incr_conc.sh
1
```

En analysant ce qu'il s'est passé, on peut remarquer que dans la version concurrente, des incrémentations n'ont pas été prises en compte. Cela vient du fait que le compteur est partagé entre les différents processus et que l'opération d'incrémentation n'est pas atomique. Une opération atomique est une opération qui a la caractéristique de s'exécuter entièrement sans être interrompue. Or l'opération d'incrémentation est composée de 3 sous opérations à savoir une lecture, une addition et une écriture. Il est donc tout a fait possible qu'un processus soit interrompu et commuté entre la lecture et la mise à jour du fichier.

Voici un exemple de scénario expliquant le résultat obtenu :



Le premier processus `incr.sh` a été commuté entre sa lecture et son écriture. De ce fait la valeur lue n'est plus la dernière valeur écrite. On peut noter aussi que sans la commande `wait` dans le script `incr_conc.sh`, la valeur 0 aurait pu être affichée si le processus père s'était exécuté sans laisser la main à ses fils.

7.2 Section critique et verrous

Nous avons illustré précédemment que la concurrence pouvait engendrer des incohérences sur les ressources partagées si aucun mécanisme de synchronisation n'était mis en place. Avant de mettre en place ces mécanismes il est nécessaire d'analyser le code exécuté concurremment et de repérer les portions de code contenant des opérations sur les ressources partagées qui ne doivent pas être interrompues par une opération concurrente et conflictuelle (qui manipule la même ressource). On appelle ces portions de code des **sections critiques**. Une portion de code identifiée comme section critique doit assurer à tout instant qu'au plus un seul processus puisse l'exécuter. On dit que les processus sont en **exclusion mutuelle**. Dans notre exemple d'incrémentation les trois opérations (lecture, addition et écriture) forment une section critique.

Pour délimiter une section critique dans le programme nous utilisons un verrou (on parle aussi de mutex). Un verrou est un objet système qui offre deux opérations atomiques :

- `lock` : qui prend en paramètre un identifiant de ressource (ex : un nom de fichier) et qui bloque tout processus tant que la ressource est occupée.
- `unlock` : qui prend en paramètre un identifiant de ressource et qui notifie que la ressource est libre permettant ainsi de débloquer éventuellement un processus qui serait bloqué sur l'opération `lock`. Les processus en attente sont débloqués suivant un ordre FIFO : le processus débloqué sera le processus qui a attendu le plus longtemps.

Il est important de noter que ces deux opérations sont indissociables dans la mesure où tout appel à `lock` doit nécessairement donné lieu dans un temps fini à un appel à `unlock`. Si ceci n'est pas respecté, des processus peuvent rester bloqués indéfiniment et ainsi ne jamais accéder à la ressource.

Un verrou se résume donc à deux variables :

- un booléen qui indique si la ressource est occupée ou pas.
- une file d'attente permettant de sauvegarder l'ordre des requêtes, i.e., la liste des processus qui ont été bloqué par l'opération `lock`.

L'algorithme des opérations `lock` et `unlock` sont les suivants

```
lock :
    si booléen = vrai alors
        insérer son id dans la file
        se mettre en attente
    fin si
    mettre le booléen à vrai
```

```
unlock :
    mettre le booléen à faux
    si la file est non vide
        réveiller le suivant
        supprimer son id de la file
    fin si
```

7.3 Synchronisation en shell

Le shell est un langage de commande permettant d'utiliser les services offerts par le système d'exploitation. Cependant ce langage est très haut niveau ce qui abstrait des fins mécanismes qui sont plus complexes à paramétrer et à manipuler. Hormis la primitive de synchronisation `wait`, le shell ne propose pas de primitive de verrouillage qui sont des mécanismes plutôt accessibles à des langages plus bas niveau comme le langage C. Par conséquent, nous allons fournir une commande `synchro` codée en C (fichier synchro.c) qui permet d'utiliser des primitives de verrouillage en shell. Cette commande prend 2 arguments : une sous-commande et un nom de verrou. Il existe 4 sous-commandes

- `create` qui permet de déclarer et d'instancier un verrou dans le système. Ceci renvoie une erreur si le nom du verrou existe déjà
- `lock` qui permet de prendre le verrou ou d'attendre tant que ce dernier est utilisé. Ceci renvoie une erreur si le verrou n'existe pas.
- `unlock` qui permet de libérer le verrou. Ceci renvoie une erreur si le verrou n'existe pas.
- `destroy` qui permet de détruire le verrou. Après cette commande le nom du verrou n'est plus accessible et tout appel ultérieur à `lock` et `unlock` engendrera une erreur. Ceci renvoie une erreur si le verrou n'existe pas.

Pour éviter certaines erreurs, il est déconseillé d'utiliser des caractères comme `/` dans les noms de verrou. Dans la plupart des cas on utilise le nom de la ressource suivi de la chaîne `.lock`.

Pour installer la commande `synchro` sur votre poste de travail, exécutez les commandes suivantes.

1. `mkdir $HOME/cmd`
2. `echo "export PATH=$PATH:$HOME/cmd" >> $HOME/.bashrc`
3. `source $HOME/.bashrc`
4. `gcc -o $HOME/cmd/synchro synchro.c -lpthread`

Si nous reprenons notre exemple sur l'incrémentation concurrente d'un compteur partagé, voici comment nous modifierions les scripts `./incr_conc.sh` et `./incr.sh`.

scripts/incr_conc_lock.sh

```
#!/bin/bash                                                    1
#incr_conc.sh                                                  2
                                                               3
```

```
fichier=/tmp/cpt
synchro create $(basename $fichier).lock
echo 0 > $fichier

./incr_lock.sh $fichier &
./incr_lock.sh $fichier &
./incr_lock.sh $fichier &

wait

synchro destroy $(basename $fichier).lock

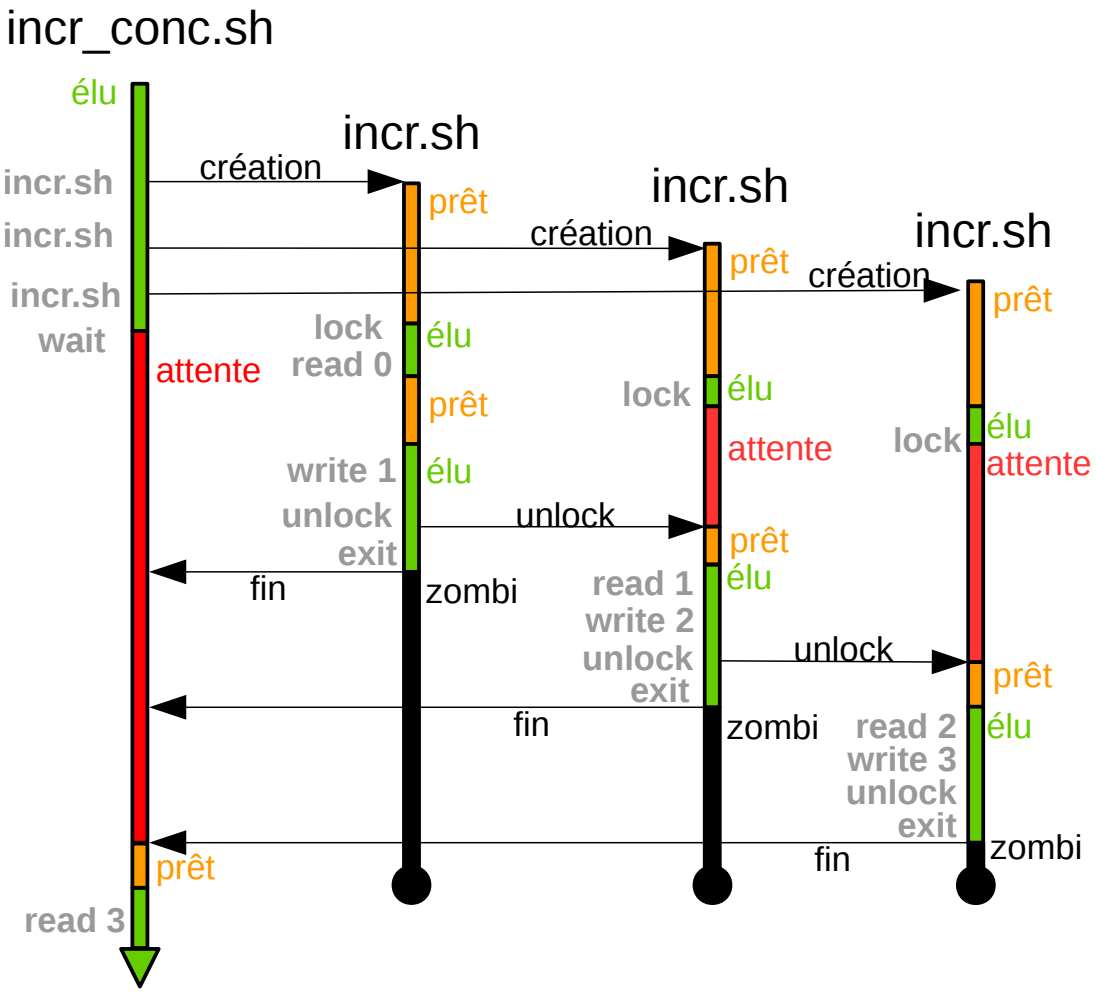
cat $fichier
```

scripts/incr_lock.sh

```
#!/bin/bash
#incr.sh

fic=$1
synchro lock $(basename $fic).lock
read val <$fic
val=$(($val + 1))
echo $val >$fic
synchro unlock $(basename $fic).lock
```

Voici un exemple de scénario d'exécution avec la mise en place de verrou.



7.4 Synchronisation à base de sémaphores

Un **sémaphore** est un objet système qui généralise la notion de verrou en autorisant qu'il y ait au plus N processus en section critique (pour un verrou N est égal à 1). Il suffit donc de remplacer le booléen du verrou par un compteur qui désigne le nombre de processus qui peut encore entrer en section critique et par conséquent :

- un processus attendra si le compteur est égal à 0.
- la mise à jour du booléen à vrai revient à décrémenter le compteur
- la mise à jour du booléen à faux revient à incrémenter le compteur

Ainsi les algorithmes de `lock` et `unlock` pour un sémaphore deviennent :

```
lock :
    si compteur = 0 alors
        insérer son id dans la file
        se mettre en attente
    fin si
    décrémenter le compteur
```

```
unlock :
    incrémenter le compteur
    si la file est non vide
        réveiller le suivant
        supprimer son id de la file
    fin si
```

Remarques : Lorsque l'on parle de sémaphore, on note souvent les opérations `lock` et `unlock` respectivement par `P` (*"**P**uis-je continuer ?"*) et `V` (*"**V**as-y !"*).

En considérant maintenant un compteur au lieu d'un booléen, il est maintenant possible de paramétrer au moment de la création du sémaphore la valeur de N. Avec la commande `synchro`

fournie, il faut préciser cette valeur en troisième argument de la sous-commande `create`.

Avec les sémaphores il est envisageable d'initialiser le compteur à 0. Ce cas permet d'implanter des synchronisations de type **barrière** qui permet de bloquer un ou plusieurs processus lorsqu'une condition n'est pas satisfaite initialement. Dans ce cas les appels à `lock` et `unlock` pour un même sémaphore peuvent être bien souvent dans deux scripts différents : un script bloquant (appel à `lock`) et un script débloquant (appel à `unlock`).

Pour illustrer ceci, nous allons implanter un mécanisme qui prend en compte N processus écrivains et 1 processus lecteur. Les processus écrivains vont écrire de manière concurrente dans un fichier leur PID et attendre que le processus lecteur aient lu tout le fichier. Le processus lecteur attendra que tous les processus écrivains aient écrit pour afficher le contenu du fichier et ensuite débloquer les lecteurs. On considérera également un processus dédié au déploiement et à la destruction des différents objets partagés.

scripts/ecrivain.sh

```
#!/bin/bash
#ecrivain.sh
usage="$0 <id> <fichier>"

if [ $# -ne 2 ]
then
       echo $usage &&  exit 1
fi

id=$1
fichier=$2

echo $$ >> $fichier #ecriture
synchro unlock lecteur #Vas-y lecteur !
synchro lock ecrivain$id #Puis-je continuer ?
```

scripts/lecteur.sh

```
#!/bin/bash
#lecteur.sh
usage="$0 <nb_ecrivains> <fichier>"

if [ $# -ne 2 ]
then
       echo $usage && exit 1
fi

nb_ecrivains=$1
fichier=$2

i=0
while [ $i -lt $nb_ecrivains ]
do
       synchro lock lecteur #Puis-je lire ?
       i=$(($i + 1))
done

```

```
cat $fichier #lecture

i=0
while [ $i -lt $nb_ecrivains ]
do
	synchro unlock ecrivain$i # Vas-y, débloque toi ecrivain i !
	i=$(($i + 1))
done
```

scripts/lecteur_ecrivain.sh

```
#!/bin/bash
#lecteur_ecrivain.sh
usage="$0 <nb_ecrivains>"

if [ $# -ne 1 ]
then
	echo $usage &&  exit 1
fi

nb_ecrivains=$1

fichier="/tmp/fic"
rm -f $fichier
synchro create lecteur 0
./lecteur.sh $nb_ecrivains $fichier &

i=0
while [ $i -lt $nb_ecrivains ]
do
	synchro create ecrivain$i 1
	./ecrivain.sh $i $fichier &
	i=$(($i + 1))
done

wait
i=0
while [ $i -lt $nb_ecrivains ]
do
	synchro destroy ecrivain$i
	i=$(($i + 1))
done

synchro destroy lecteur
```

Annexe A

Aide mémoire shell

2I010 - Aide-mémoire Shell

Commandes de base

pwd	permet de connaître le chemin absolu du répertoire courant (celui dans lequel on se trouve).
cd [chemin]	positionne le répertoire courant à "chemin" ou au répertoire d'accueil (HomeDirectory) si aucun paramètre
ls [opt] [sel]	liste le contenu d'un répertoire ou affiche les informations d'un fichier passé en paramètre. Un certain nombre d'options peuvent être indiquées (opt) dont : **-l** affiche des informations relatives aux éléments (type, droits, nombre de blocs, taille, date...). **-a** visualise tous les éléments, y compris les fichiers et répertoires cachés (sous Unix, les éléments cachés commencent par le caractère ".")

Redirection des E/S

> fichier	redirige la sortie standard d'une commande dans fichier (écrase le contenu précédent du fichier).
>> fichier	redirige la sortie standard d'une commande dans fichier (ajoute à la fin du fichier).
< fichier	redirige l'entrée standard de la commande depuis fichier.

Meta-caractères

*	signifie "n'importe quelle séquence de caractères".
?	signifie "n'importe quel caractère".
[wtz]	signifie "le caractère **w** ou le caractère **t** ou le caractère **z**".
[a-p]	signifie "n'importe quel caractère entre **a** et **p**".

Séparateurs de commandes

cmd1 ; cmd2	permet de séparer deux commandes lorsqu'elles sont sur la même ligne. Elles sont exécutées séquentiellement.
cmd1 | cmd2	"tube" (*pipe*) de communication entre deux processus : connecte la sortie standard de la commande **cmd1** à l'entrée standard de la commande **cmd2**.
cmd &	exécute la commande **cmd** en arrière plan.

Substitution de variables et de commande

$#	indique le nombre de paramètres de la ligne de commande.
$n	avec n compris entre 0 et 9, indique la valeur du nième paramètre. $0 correspond au nom de la commande invoquée. $n correspond à la chaîne vide s'il y a moins de n paramètres.
"$@"	donne l'ensemble des paramètres en préservant les espaces.
$$	l'identifiant (PID) du processus en cours d'exécution.
$PPID	l'identifiant (PID) du processus père du processus en cours d'exécution.
$!	l'identifiant (PID) du dernier processus lancé en arrière plan.
$?	la valeur de retour de la dernière commande exécutée en premier plan.
$(cmd)	substitue à **$(cmd)** le texte produit sur la sortie standard par l'exécution de la commande **cmd** (équivaut à **`cmd`**)
$((calc))	substitue à **$((calc))** le résultat de l'évaluation de l'expression numérique **calc** (équivaut à **`expr calc`**)

Protection de caractères

\x	un backslash protège n'importe quel caractère ayant normalement un sens particulier pour le shell. Par exemple, ***** est interprété comme le caractère * et non comme le méta-caractère *. \\ correspond au caractère \.
"xyz"	protège la chaîne xyz du découpage sur les espaces lors de la séparation des arguments. Empêche l'interprétation des séparateurs de commandes, des méta-caractères et des opérateurs de redirection. Par contre, les variables et les commandes entre backquotes sont substituées.
'xyz'	protège la chaîne xyz du découpage, comme les guillemets, mais n'effectue **aucune** substitution. Aucun caractère n'a de sens particulier dans une chaîne entre apostrophes, même pas \.

Structures de contrôle dans un script shell

Structure conditionnelle la structure conditionnelle **if** du shell repose sur la valeur de retour d'une commande
ATTENTION : contrairement au C la valeur 0 est considérée comme vrai (faux si ≠ 0)

```
if  une_commande
then
        séquence d'instructions
elif  une_commande
then
        séquence d'instructions
else
        séquence d'instructions
fi
```

Commande de test le shell fournit la commande **test** qui retourne 0 si **xxx** est vrai ; ainsi que l'alias **[:**

test xxx équivaut à **[xxx]**

Tests sur les chaînes de caractères	`[ -z "ch1" ]`	ch1 est une chaîne vide
	`[ "ch1" = "ch2" ]`	ch1 = ch2
	`[ "ch1" != "ch2" ]`	ch1 ≠ ch2
Tests sur les valeurs numériques	`[ e1 -lt e2 ]`	e1 < e2
	`[ e1 -gt e2 ]`	e1 > e2
	`[ e1 -le e2 ]`	e1 ≤ e2
	`[ e1 -ge e2 ]`	e1 ≥ e2
	`[ e1 -eq e2 ]`	e1 = e2
	`[ e1 -ne e2 ]`	e1 ≠ e2
Tests sur l'arborescence de fichiers	`[ -e "ch1" ]`	il existe un fichier ou répertoire ch1
	`[ -f "ch1" ]`	il existe un fichier ch1
	`[ -d "ch1" ]`	il existe un répertoire ch1
Opérations logiques (évaluation progressive)	`test_1 || test_2`	test_1 **ou** test_2
	`test_1 && test_2`	test_1 **et** test_2
	`! un_test`	négation de un_test

boucle for à chaque tour de boucle le shell affecte à une variable (**déclarée sans $**) une des valeurs de la liste :

```
for nom_var in liste_de_valeurs
do
   séquence d'instructions pouvant utiliser $nom_var
done
```

boucle while comme pour le **if**, la boucle "tant que" considère la valeur de retour d'une commande qui peut être **test** (**[xx]**)

```
while une_commande
do
   séquence d'instructions
done
```

case en shell le branchement multiple (équivalant d'un *switch*) recherche des patterns (séquence de caractères pouvant contenir des méta-caractères comme ***** ou **?**). Contrairement au C, il ne considère pas des intervalles de valeurs
Attention : la séquence d'instructions qui suit doit forcément se terminer par les caractères **;;**

```
case $var in
   pattern) séquence d'instructions;;
esac
```

Quelques commandes shell

cat fich	affiche le contenu du fichier fich sur la sortie standard.
cp f1 f2	effectue la copie du fichier f1 dans le fichier f2. Si f2 existait déjà, son ancien contenu est perdu.
echo	affiche ses arguments sur la sortie standard.
exit [n]	interrompre l'exécution d'un script avec la valeur de retour n (0 si pas de problème, ≠0 sinon).
export var	crée une copie de la variable var dans le contexte d'exécution de tous les nouveaux processus de la descendance du processus courant.
expr expression	calcule l'expression arithmétique indiquée en argument. Attention, ne travaille qu'en arithmétique entière.
grep [opt] str fich	affiche sur la sortie standard les lignes de fich contenant l'expression str. Si l'option **–v** est précisée, les lignes ne contenant pas l'expression str sont affichées. Si aucun nom de fichier n'est donné, fait la recherche à partir de l'entrée standard. L'option **-R** permet de faire une recherche récursive dans un répertoire. Le caractère **^** indique le début de ligne et le caractère **$** indique la fin de ligne.
find [start] [crit]	recherche les fichiers ou répertoires correspondant aux critères passés en paramètre : **-name '*.sh'** pour un nom finissant pan sh, ***-size +100k*** pour un fichier de plus de 100k, **-type f** pour un fichier, ….
kill -no pid	envoie le signal no (no est un numéro ou un nom de signal) au processus d'identifiant pid. **kill –l** donne la liste des signaux.
mkdir dir	crée le répertoire dir dans le répertoire courant.
mv f1 f2	déplace le fichier f1 dans le fichier f2. Si f2 existait déjà, son ancien contenu est perdu. Après l'opération, f1 n'existe plus. Cette commande peut aussi s'appliquer à des répertoires.
read var	range dans la variable var la ligne de caractères lue sur l'entrée standard.
rm f1	détruit le fichier f1.
rmdir dir	détruit le répertoire dir, à condition que celui-ci soit vide.
set -x	toute commande suivant cette instruction sera tracée sur la sortie erreur standard.
shift [n]	décale d'un cran les paramètres du script (pas de valeur n) ou bien de "n crans" (si n est précisé). Attention: comportement non standard si le nombre de paramètres n'est pas suffisant.
source script.sh	exécute les instructions de script.sh dans le processus courant.
touch fic1 fic2	change la date de modification des fichiers donnés en paramètre. Si les fichiers n'existent pas, les crée.
tr seq1 seq2	affiche sur la sortie standard le texte reçu de l'entrée standard en substituant le nième caractère de seq1 par le nième caractère de seq2.
trap [arg] signal	exécute arg lors du traitement de signal. Si arg est une chaîne vide, le signal est ignoré, si arg est absent, le traitement par défaut du signal est rétabli.
wait [n …]	bloque le processus courant tant que les fils dont les identifiants sont passés en paramètre ne sont pas terminés. Si aucun paramètre n'est donné, le processus attend la fin de tous ses fils.

Annexe B

Guide d'installation de Linux Ubuntu sur un ordinateur personnel

Ubuntu est issue de la distribution mère Debian et est une des distributions les plus utilisées par les utilisateurs grand public. Cette annexe décrit comment installer Linux Ubuntu sur un ordinateur personnel ayant Windows comme OS d'origine.

B.1 Téléchargement de l'image Iso

L'installation d'un système d'exploitation nécessite un fichier iso qui peut être vu comme une archive d'un CD-ROM.

Tâche(s) à effectuer :

1. Télécharger l'image iso Ubuntu sur cette page web :
   `https://ubuntu-fr.org/telechargement`

Une fois l'image téléchargée, il y a deux alternatives :

- soit faire une installation native, dans ce cas il est nécessaire d'avoir un support de stockage externe bootable comme une clé usb ou un CD-ROM et de suivre les instructions de l'exercice B.2.
- soit vous souhaitez faire une installation dans une machine virtuelle, dans ce cas le fichier iso téléchargé suffira et vous pouvez passer directement à l'exercice B.3.

B.2 Installation native de Ubuntu en double boot

L'installation native permet d'utiliser Linux Ubuntu à la place du système d'exploitation d'origine (Windows, MacOS ou bien même une autre distribution Linux). Si vous souhaitez faire cohabiter Linux Ubuntu et votre système d'exploitation d'origine sur la même machine, il est possible de faire un double boot. Le double boot permet de choisir lors du démarrage de la machine, le système d'exploitation désiré à travers un utilitaire (ici c'est l'utilitaire GRUB). Cette solution a l'avantage d'utiliser directement les ressources de la machine (sans couche de virtualisation intermédiaire), on gagne ainsi en performance de manière significative. Cependant cette solution reste très technique et s'adresse à des utilisateurs avertis (il y a un risque de dysfonctionnement de la machine si l'installation se passe mal) et il n'est pas possible d'utiliser les deux systèmes d'exploitations en même temps.

Tâche(s) à effectuer :

1. Produire au préalable un support de stockage externe bootable. Pour rendre une clé USB bootable, il est possible d'utiliser le logiciel Rufus. Pour plus d'informations, se référer aux nombreux tutoriels disponibles sur le web.
2. Créer une partition sur le disque dur pour pouvoir y installer Ubuntu. Il faut prévoir au moins 60 Go pour l'installation de l'OS et des différents logiciels. Pour ce faire dans Windows (10), aller sur l'utilitaire "Gestion du disque ". Le "Disque 0" représente le disque dur de la machine. Les ordinateurs avec plusieurs disques durs sont donc notés Disque 0, Disque 1, etc. La partie "Réservé au système" est nécessaire au démarrage de Windows : il ne faut pas y toucher. La partie indiquée "C :" et Sain (Démarrer, Fichier d'échange, Vidage sur incident...) est la partition visible dans le poste de travail, là où sont stockés le système Windows et les fichiers de l'utilisateur (documents, images, musique, etc).
   - Faire un clic droit sur un des disques et sélectionner "Réduire le volume".
   - Indiquer une valeur dans la Quantité d'espace à réduire (en Mo), par exemple ici 20000 va réduire notre partition de 20Go. Dans un autre exemple, si le disque est de 500Go et que vous ne souhaitez que garder 100Go pour le système, il va falloir réduire le volume de 400000 (400 000Mo soit environ 400Go). La « Taille totale en Mo après réduction » indique la taille du disque après modification.

   NB : il n'est pas nécessaire de formater l'espace disque libéré, ceci sera fait lors de l'installation d'Ubuntu.
3. Insérer le support stable bootable et redémarrer l'ordinateur. Pour que la machine boot sur le support stable, il faut s'assurer dans le BIOS que ce dernier se situe avant la partition du système d'exploitation d'origine dans la séquence de recherche de partition bootable. Pour accéder au BIOS, il faut au moment du démarrage, pour la plupart des cartes mères soit appuyer sur la touche *F2* soit appuyer sur la touche *Suppr*.
4. Suivre les différentes instructions d'installations. À un moment, il sera demander à quel endroit du disque Ubuntu doit être installer. Sélectionner l'espace disque libéré précédemment et continuer l'installation.
5. Au redémarrage de l'ordinateur lorsque l'installation est terminée penser à enlever le support bootable de la machine. Il sera ensuite possible de choisir quel système d'exploitation utiliser.

Si vous avez opté pour cette solution, passez directement à la section B.4

B.3 Installation de Ubuntu dans une machine virtuelle

La virtualisation de machine permet d'installer et d'utiliser plusieurs systèmes d'exploitations en même temps. La virtualisation passe donc par l'utilisation d'un logiciel appelé hyperviseur qui permet d'émuler une machine physique. On peut donc remarquer qu'il existera le système d'exploitation d'origine (ex : Windows) qui sera le système d'exploitation dit "hôte" ou "host" (c'est à dire le système réel de l'ordinateur physique) dans lequel nous allons installer un hyperviseur de machines virtuelles qui permettra d'accueillir et d'exécuter plusieurs systèmes dit "invités" ou "guest". Dans le cadre de ce guide nous utiliserons l'hyperviseur VirtualBox. Cette solution a l'avantage de ne pas toucher à la configuration physique initiale de la machine et de pouvoir utiliser plusieurs systèmes d'exploitation en même temps mais qui peut présenter des pertes significatives de performances (le

système hôte étant toujours prioritaire par rapport aux invités).

Tâche(s) à effectuer :

1. Télécharger VirtualBox correspondant à votre système hôte et votre architecture (i386 pour les processeurs intel et AMD64 pour les processeurs AMD) ainsi que l'extension pack sur cette page web : `https://www.virtualbox.org/wiki/Downloads`
2. Installer virtualBox ainsi que l'extension pack
3. Ouvrir VirtualBox et créer une nouvelle machine :
   - lui donner un nom (par exemple "Ubuntu") et choisir le système d'exploitation souhaité : dans notre cas le type sera Linux et la version sera Ubuntu (64 bits). Cliquez sur *suivant*
   - Choisir la quantité de mémoire RAM à affecter sur cette machine (la mémoire RAM affectée sera la quantité de mémoire mise à disposition de l'OS invité). Il est conseillé de mettre au maximum de la zone verte. Cliquez sur *suivant*
   - Créer une nouvelle image disque, c'est ce qui émulera un disque dur pour le système invité, mais ça sera un fichier du point de vue du système hôte. Choisir le type VDI et choisir votre mode d'allocation (dynamique ou fixe), puis créer le disque.

Maintenant que la machine est créée, il est maintenant possible d'installer Ubuntu.

Tâche(s) à effectuer :

1. Sélectionner la machine virtuelle créée et cliquer sur "configuration". Dans l'onglet "Stockage", créer un nouveau contrôleur Sata ou IDE (pour émuler un lecteur CD-ROM) et y monter l'image iso d'Ubuntu téléchargée précédemment.
2. Dans l'onglet "Système", configurer si nécessaire d'autres paramètres physiques comme par exemple le nombre de cœurs de calcul mise à disposition du système invité.
3. Démarrer la machine virtuelle en cliquant sur le bouton start.
4. Suivre les différentes instructions d'installations.
5. Une fois l'installation terminée, on peut remarquer que la taille de la fenêtre de la machine virtuelle (qui émule l'écran) n'est pas extensible pour la mettre à plein écran de la machine physique. S'assurer avant cela d'avoir installer l'extension pack de virtualbox puis cliquer sur "périphérique" dans la barre d'outil Virtual Box de la fenêtre, puis sur "insérer l'image CD des additions invité".
6. Exécuter l'installation des additions. Ceci nécessite d'avoir les droits administrateur, entrer le mot de passe dès que l'on y est invité.
7. Redémarrer la machine virtuelle et s'assurer que tout fonctionne

B.4 Installation des logiciels de base sur Ubuntu

Avant toute chose il faut s'assurer que la machine (physique et virtuelle le cas échéant) ait bien accès à Internet (tester la connexion en ouvrant un navigateur par exemple).

Tâche(s) à effectuer :

1. Ouvrir l'application *terminal* et mettre à jour le système (ceci peut prendre un moment). Pour cela, taper les deux commandes suivantes qui nécessitent de renseigner le mot de passe de l'utilisateur :

   ```
   sudo apt-get update
   sudo apt-get upgrade
   ```

   Un redémarrage est potentiellement nécessaire.

2. Installer Java 8 en tapant :

   ```
   sudo apt-get install openjdk-8-jdk
   ```

3. Vérifier l'installation en tapant :

   ```
   java -version
   ```

4. Installer le serveur ssh en tapant :

   ```
   sudo apt-get install openssh-server
   ```

5. Vérifier l'installation en tapant :

   ```
   ssh localhost
   ```

   L'installation s'est bien passé si il est demandé de taper le mot de passe de l'utilisateur.

6. Fermer le terminal.